

Full Length Article

Basis sharing and coefficient pruning for efficient neural network compression

Van Tien Pham 

Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam

ARTICLE INFO

Keywords:

Weight sharing
Structured pruning
Network compression

ABSTRACT

Network compression is essential for deploying deep neural networks on resource-constrained devices, yet existing approaches often focus on either filter pruning or weight sharing in isolation. In this work, we propose a unified compression framework, SaP, that jointly leverages basis sharing and coefficient pruning to reduce redundancies in both convolutional filters and their resulting feature maps. By representing filters as linear combinations of a reduced set of shared basis filters and removing redundant coefficients, SaP achieves higher compression ratios while preserving discriminative information. This hybrid design mitigates the bottleneck problem inherent in sole basis-sharing methods, reduces computational complexity, and enables efficient fine-tuning in the coefficient subspace. We further provide a theoretical analysis showing that, under the orthonormal basis induced by the singular value decomposition, distances between filters are preserved in the coefficient space, justifying redundancy analysis and pruning in this low-dimensional representation. Extensive experiments on image classification, object detection, instance segmentation, and keypoint detection demonstrate that SaP consistently outperforms state-of-the-art baselines in terms of compression, accuracy retention, and practical efficiency. Ablation studies further validate the effectiveness of the joint approach and provide insights into hyperparameter selection and feature preservation. Our results highlight the potential of combining complementary compression strategies to create highly efficient, deployable neural networks.

1. Introduction

Network compression is a pivotal technique in machine learning aimed at reducing the computational and memory requirements of neural networks. The goal is to create more efficient models while maintaining comparable performance, facilitating deployment on resource-constrained devices, and enhancing overall scalability and generality. Various methods are employed to achieve compression, including pruning, quantization, knowledge distillation, low-rank decomposition, neural architecture search (NAS), and weight sharing (Kumar et al., 2025; Li et al., 2026; Pham et al., 2025b, 2026).

Among these diverse paradigms, structured pruning (Pham et al., 2025b; Tofigh et al., 2026a) has been extensively explored, whereas weight sharing (Kang & Kim, 2021; Li et al., 2019) remains comparatively underinvestigated. Both techniques, however, share several conceptual similarities from a model compression standpoint. From a motivation perspective, both rely on the hypothesis that deep networks are over-parameterized and thus contain redundant information that can be eliminated, either by removing redundant weights (structured pruning) or by representing them more compactly (weight sharing) (Pham et al., 2026). From a deployment perspective, both approaches yield dense,

hardware-friendly networks that can be executed on standard devices without specialized accelerators (unlike unstructured sparsification or quantization) (Pham et al., 2025a). Interestingly, the strengths of these two methods are complementary rather than overlapping and can therefore be jointly leveraged for enhanced compression (Shin et al., 2024). Despite this potential synergy, they have typically been developed in isolation, and their integration remains largely unexplored in the literature.

In this research, we revisit convolutional neural networks and make two key observations. First, all filters within a convolutional layer share identical dimensions and operate on the same input tensor to produce their respective feature maps, as illustrated in the top part of Fig. 1. Second, redundancy frequently arises among these filters, as widely reported in similarity-based pruning studies (Li et al., 2025; Pham et al., 2024a). Because partially similar filters extract overlapping information from the same input, their output features often contain redundant components. For instance, in Fig. 1, filters $\mathcal{W}_1$ and $\mathcal{W}_3$ generate output maps $\mathcal{O}_1$ and $\mathcal{O}_3$ that share a similar (red) region, which is computed twice. This duplication reflects redundancy in both the filter space and the resulting feature space. Conventional pruning removes an entire filter (e.g., $\mathcal{W}_3$) to eliminate redundancy, but this may also discard valuable

E-mail address: phamvantien@tdtu.edu.vn<https://doi.org/10.1016/j.neunet.2026.109630>

Received 21 November 2025; Received in revised form 24 July 2026; Accepted 8 September 2026

Available online 14 September 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

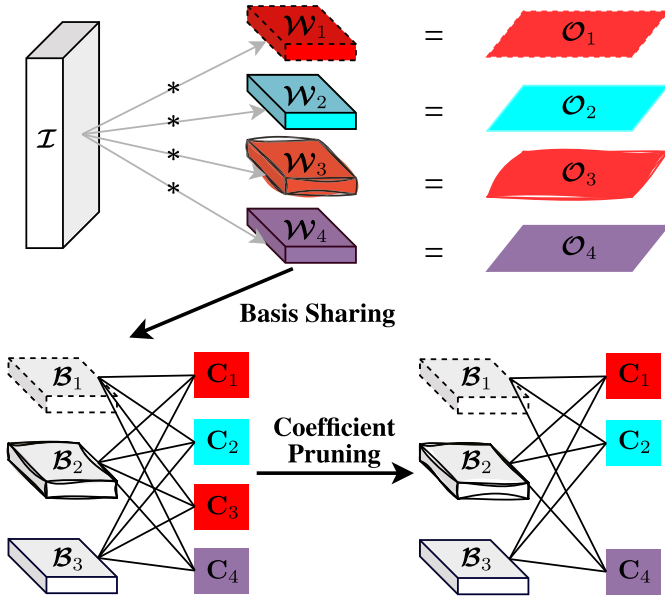


Fig. 1. Redundancy in convolutional filters results in duplicated computation of similar features (top). The proposed SaP framework mitigates this inefficiency by sharing basis filters (bottom left) and pruning similar coefficients (bottom right).

information, since the two filters are only partially similar. An effective compression strategy should therefore address redundancies in both spaces while retaining the essential discriminative information of the original filters.

Motivated by these observations, we propose a unified compression framework that combines basis sharing and coefficient pruning to jointly reduce redundancies in convolutional filters and feature maps. Since all filters have identical shapes, they can both theoretically (due to shared dimensions) and empirically (due to redundancy) be represented as linear combinations of a reduced set of shared basis filters. For example, in the bottom left of Fig. 1, the four original filters $\{\mathcal{W}_1, \mathcal{W}_2, \mathcal{W}_3, \mathcal{W}_4\}$ can be reconstructed from only three basis filters $\{\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3\}$ linearly combined with four corresponding coefficient vectors $\{C_1, C_2, C_3, C_4\}$. This representation is more compact, as the total number of parameters in the basis filters and coefficients is smaller than that of the original filters. However, redundancy may still persist in the coefficient space, e.g., similar coefficients C_1 and C_3 lead to overlapping output features $\mathcal{O}_1$ and $\mathcal{O}_3$. To further eliminate this inefficiency, we perform structured pruning directly in the coefficient space by removing redundant coefficients such as C_3 , which also eliminates the corresponding output feature map $\mathcal{O}_3$, as shown in the bottom right of Fig. 1.

This hybrid approach offers several advantages. First, it removes redundancies across both filter and feature domains, extending the compression potential beyond either weight sharing or pruning alone. In weight sharing (Kang & Kim, 2021), the compression ratio is typically controlled only by the number of bases, whereas in the hybrid approach, additional compression is achieved by removing redundant coefficients that remain fixed in conventional weight sharing. This design is also more flexible than directly removing entire filters, as done in standard pruning methods (Lin et al., 2025; Tang et al., 2025). Second, for a pre-defined compression target, the hybrid design yields a more balanced architecture. Compared to pure filter pruning (Li et al., 2025; Tofigh et al., 2026a), it avoids excessive removal of informative filters, while compared to sole basis sharing (Kang & Kim, 2021; Li et al., 2019), it further eliminates redundancy in the coefficient space. An additional benefit is the mitigation of the bottleneck problem inherent in basis sharing, where the number of basis filters is much smaller than the number of coefficients. This imbalance can hinder training (Shin et al., 2024), and

pruning redundant coefficients alleviates this issue by reducing the number of active coefficients. Third, operating in the coefficient subspace substantially reduces computational complexity and enables faster pruning and fine-tuning procedures. This is because the pruning operations are performed on the coefficient vectors, which are significantly smaller in size compared to the original filters, resulting in fewer parameters to evaluate during the pruning process. Moreover, under the orthonormal basis induced by truncated SVD, distances between convolutional filters are exactly preserved in the coefficient space, providing a formal justification for performing redundancy analysis and pruning in this reduced representation without loss of geometric fidelity. After removing redundant coefficients, we freeze the shared basis filters and only update the remaining coefficient vectors via a lightweight fine-tuning process. Unlike existing hybrid decomposition–pruning approaches (Li et al., 2020b; Pham et al., 2025a; Ruan et al., 2021; Shin et al., 2024; Sui et al., 2025; Wong et al., 2024), which typically prune decomposed layers or jointly optimize basis filters and coefficients during retraining, SaP performs redundancy-aware structured pruning directly in the coefficient space induced by a shared orthonormal basis and freezes the shared basis filters during fine-tuning, resulting in more efficient optimization and improved compression efficiency. Consequently, this approach not only accelerates pruning but also facilitates iterative optimization and fine-tuning, making it highly practical for large-scale networks and real-time deployment scenarios.

Our main contributions are summarized as follows:

- We propose the joint use of weight sharing and structured pruning, analyzing how these two complementary strategies can be integrated to reduce both filter- and feature-level redundancy.
- We introduce SaP, a concrete implementation of this concept, composed of two modules: a truncated singular value decomposition-based basis sharing mechanism and a redundancy-aware coefficient pruning algorithm that removes redundant coefficients while preserving filter diversity.
- We provide a theoretical analysis proving that, under the orthonormal basis induced by SVD, distances between convolutional filters are exactly preserved in the coefficient space. This result provides a formal justification for performing redundancy analysis and pruning in a low-dimensional representation without loss of geometric fidelity.
- We conduct extensive experiments across diverse vision tasks, including image classification, object detection, instance segmentation, and keypoint detection, demonstrating the consistent superiority of SaP over state-of-the-art baselines across convolutional neural networks, transformer, and hybrid convolution–transformer architectures.
- We further provide ablation studies and discussions on key aspects such as hyperparameter selection, compression efficiency, practical latency, qualitative feature preservation, and statistical robustness.

2. Related works

Model compression has been extensively studied to reduce the computational and memory requirements of deep neural networks while maintaining accuracy. Existing approaches can be broadly categorized into pruning, quantization, knowledge distillation, low-rank decomposition, weight sharing, and NAS. We briefly review the most relevant methods and highlight how our proposed SaP framework differs from them.

2.1. Pruning

Pruning aims to remove redundant or unimportant parameters from a model. It can be categorized by granularity into unstructured and structured pruning. Early unstructured methods removed individual weights, achieving high compression but producing irregular sparsity patterns that are inefficient for deployment (Pham et al., 2024a).

In contrast, structured pruning removes entire filters, channels, or blocks (Pham et al., 2025b), yielding dense and hardware-friendly models that can be directly deployed on standard devices without specialized support (Li et al., 2026). The coefficient pruning proposed in this work belongs to the structured category, as it removes full rows of the coefficient matrix, effectively pruning corresponding output channels.

The pruning criterion, which determines which components to remove, serves as the foundation of structured pruning. Existing criteria can be broadly grouped into magnitude-based (Chen et al., 2022; Lin et al., 2020a; Tang et al., 2025), similarity-based (Li et al., 2025; Pham et al., 2024a; Tofigh et al., 2026a), optimization-based (Gao et al., 2024; Hou et al., 2024; Zhang & Freris, 2024), graph-based (Chen et al., 2024c; Lee & Lee, 2026; Li et al., 2024), clustering-based (Chang et al., 2023; Chen et al., 2024a; Lin et al., 2025), and architecture search-based (Su et al., 2023; Wu et al., 2024; Zhang et al., 2025) families. For instance, Tang et al. (2025) evaluated filter importance using feature-map entropy and pairwise distances, while ACLI (Tofigh et al., 2026a) leveraged inter-layer dependencies through weak submodularity. UDSP (Gao et al., 2024) unified dynamic and static channel pruning via bi-level optimization. RGP (Chen et al., 2024c) formulated pruning as a one-shot graph-regularized optimization, and HSC (Lin et al., 2025) used high-order spectral clustering for redundancy identification. Finally, ATO (Wu et al., 2024) employed a controller network to automate pruning through architecture generation. Although effective, existing pruning approaches still face challenges. Magnitude-based methods ignore inter-filter dependencies, while data-driven criteria often require costly evaluations and are sensitive to data distribution (Pham et al., 2025b). Optimization-based and NAS-driven methods, though powerful, tend to be computationally expensive (Tang et al., 2025).

2.2. Weight sharing

Weight sharing (or parameter sharing) techniques compress models by reusing parameters across multiple elements of a network, such as scalars, kernels, filters, or layers. When scalar weights are clustered into several groups, the approach corresponds to vector quantization, whereas reusing the parameters of entire layers is often referred to as parameter tying. While these methods substantially reduce storage requirements, they typically do not lower computational complexity during inference (Kang & Kim, 2021; Shin et al., 2024).

In this work, we focus on basis sharing, which represents multiple filters or kernels as linear combinations of a shared set of basis filters (Li et al., 2020a, 2019; Shin et al., 2024; Wong et al., 2024; Yang et al., 2020, 2019). Li et al. (2019) first introduced this concept by expressing convolutional filters as combinations of filter bases. Kang and Kim (2021) later observed that recursive basis sharing across layers could cause vanishing or exploding gradients; enforcing orthogonality among basis filters mitigated this issue and improved gradient flow during training. LegoNet (Yang et al., 2019) constructed convolutional filters by assembling low-dimensional "Lego" bases, while FSNet (Yang et al., 2020) proposed a filter summary representation that treats each filter as a one-dimensional segment unwrapped from its three-dimensional form, thereby enforcing parameter sharing across filters. PENNI (Li et al., 2020a) extended basis sharing by alternately optimizing basis kernels and sparse coefficients within convolutional layers. More recently, G-SharP (Shin et al., 2024) introduced globally shared kernels that span all convolutional layers, facilitating consistent sharing throughout the network. SCWC (Li et al., 2022) further exploited the distributive property to reduce computation by clustering channels using K-means, while GAP (Chen et al., 2024a) employed nonlinear dimensionality reduction to group convolutional filters of each layer into equally sized sets for automatic kernel fusion. Wong et al. (2024) used singular value decomposition for basis scaling and then adopted the Taylor approximation to perform double pruning, i.e., to prune both the input and output channels of the bases.

Another form of parameter sharing is achieved through joint low-rank approximation, where factor matrices are shared among filters or layers. Sun et al. (2021) observed that weight tensors across layers often contain common components and proposed to represent them using shared factor matrices via coupled tensor train decomposition. Bi-JSVD (Chen et al., 2023) projected weights with similar structures from repeated modules into a shared subspace, enabling parameter reuse. Phan et al. (2024) established a link between block term decomposition with shared parameters and looped chain tensor networks by enforcing identical core tensors. CoDeC (Pham et al., 2026) combined clustering and coupled filter decomposition to enable the sharing of a common factor matrix across similar filters. GCPNet (Zhang et al., 2025) unified pruning, weight sharing, and NAS by sharing subnet weights through a bilateral coupled sampling strategy, performing forward and backward passes in each training iteration.

2.3. Other approaches

Low-rank decomposition leverages the inherent redundancy in convolutional filters by factorizing weight tensors into lower-dimensional components (Dai et al., 2025; Liebenwein et al., 2021). Tensor and matrix decomposition methods such as Tucker decomposition, canonical polyadic decomposition, and SVD have been successfully applied to deep networks (De Jonghe et al., 2026; Liebenwein et al., 2021; Tokcan et al., 2026). These methods reduce the parameter count by approximating the original filters with a set of factor matrices of smaller size. NAS focuses on automating the design of neural network architectures, rather than relying on expert knowledge and manual tuning (Chen et al., 2024b; Kumar et al., 2025). A typical NAS framework consists of three main components: the search space, the search strategy, and the evaluation method (Ganjdanesh et al., 2024; Lee & Song, 2024). Knowledge distillation aims to transfer the knowledge from a large, high-capacity model (referred to as the teacher) to a smaller, more efficient model (known as the student) (Hou et al., 2024; Zhang & Liu, 2025). The core idea is that the student can learn to mimic the behavior of the teacher by matching its outputs or internal representations, thereby retaining much of its performance while benefiting from reduced computational and memory requirements (Li et al., 2025; Lu et al., 2023).

Recent works have explored hybrid strategies that combine multiple compression paradigms (Pham, 2025). For example, FNP (Lee & Lee, 2026) integrated NAS with pruning, while HALOC (Xiao et al., 2023) combined Tucker decomposition with NAS to jointly optimize efficiency and performance. Knowledge distillation is also combined with pruning in several works (Lee & Song, 2024; Li et al., 2025; Zhang & Liu, 2025). For example, BFP (Zhang & Liu, 2025) used knowledge distillation to compensate for the loss of performance caused by pruning, while PEEL (Hou et al., 2024) performed pruning via resource reallocation and distilling the probabilistic knowledge from the trained unpruned network.

Hybrid methods that combine low-rank decomposition and pruning have recently emerged as a promising direction for leveraging the complementary strengths of both strategies (Li et al., 2020b; Pham et al., 2024b, 2025a; Ruan et al., 2021; Shin et al., 2024; Sui et al., 2025; Wong et al., 2024). Li et al. (2020b) analyzed the relationship between filter pruning and decomposition and introduced sparsity-inducing matrices to jointly guide both processes. Ruan et al. (2021) decomposed each convolutional layer into two consecutive layers and imposed regularization on the coefficient matrix to encourage the formation of a low-rank basis with sparse channels. Wong et al. (2024) applied singular value decomposition (SVD) followed by Taylor-based pruning on the resulting layers to improve inference efficiency. Shin et al. (2024) proposed to jointly learn basis filters and coefficients during training and introduced a batch-normalization-based pruning strategy to alleviate the representational bottleneck problem. Pham et al. (2024b, 2025a) employed canonical polyadic decomposition (CPD) and proposed CPDBlock pruning based on principal angles between subspaces while exploiting the

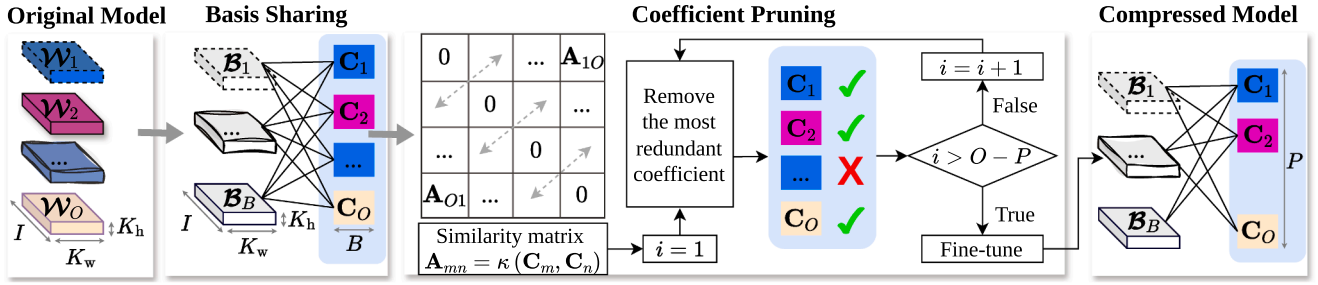


Fig. 2. Overview of the proposed SaP framework for compressing a convolutional layer. The pipeline consists of two main stages: **basis sharing** (left), which approximates filters using shared basis filters and coefficient vectors, and **coefficient pruning** (right), which removes redundant coefficients based on similarity before fine-tuning to obtain the compressed model. This two-stage process jointly reduces redundancy in both the filter space and the feature space.

essential uniqueness property of CPD. More recently, Sui et al. (2025) used tensor-train decomposition to approximate convolutional weight tensors into smaller core tensors and applied ℓ_1 -norm-based criteria for filter pruning.

Unlike these approaches, which typically apply decomposition and pruning sequentially or operate directly on decomposed layers or factor matrices, the proposed SaP framework integrates weight sharing and structured pruning within a unified formulation. Specifically, SaP performs redundancy-aware structured pruning directly in the coefficient space induced by a shared basis decomposition, enabling simultaneous removal of both filter-level and feature-level redundancies. Operating in this low-dimensional coefficient space reduces the cost of redundancy analysis while preserving the geometric structure of the original filters through the orthonormal basis representation. After removing redundant coefficients, SaP freezes the shared basis filters and updates only the remaining coefficient vectors through a lightweight fine-tuning stage. This strategy further improves compression efficiency, stabilizes the post-pruning optimization process, and reduces training overhead compared with approaches that jointly optimize both basis filters and coefficients during training. As a result, the proposed framework achieves more balanced compression while maintaining high representational fidelity under aggressive compression, leading to superior accuracy–efficiency trade-offs compared with existing decomposition–pruning hybrid methods.

3. The SaP framework

In this section, we present SaP, our proposed framework for neural network compression that jointly exploits basis sharing and coefficient pruning to eliminate redundancy at both the filter and feature levels. Fig. 2 illustrates the SaP pipeline for a single convolutional layer. Although the procedure is applied to all layers in parallel, we describe it here for one layer for clarity. Starting from the original convolutional filters, the **basis sharing** stage approximates the filters using a reduced set of shared basis filters together with their corresponding coefficient vectors, thereby capturing the common structure across filters while reducing redundancy in the filter space. Based on this representation, the **coefficient pruning** stage constructs a similarity matrix between coefficient vectors and iteratively removes the most redundant ones, which leads to the elimination of redundant output feature maps and further reduces computational cost. Finally, the compressed layer is fine-tuned to restore predictive performance.

We begin by introducing the notations and mathematical preliminaries used throughout this work (Subsection 3.1). We then describe the basis sharing module (Subsection 3.2), which approximates each filter as a linear combination of a small set of shared basis filters. Next, the coefficient pruning module (Subsection 3.3) identifies redundant filters by analyzing pairwise similarities between coefficient vectors in a low-dimensional space and prunes the least representative ones using a redundancy-aware selection algorithm.

Subsequently, we analyze the complexity of the resulting compressed model (Subsection 3.4), deriving closed-form expressions for parameter and computational reduction ratios as functions of the number of basis filters and preserved coefficients. Finally, the entire pipeline is summarized in Subsection 3.5, which details the complete SaP algorithm combining basis sharing, coefficient pruning, and fine-tuning to obtain the final compact model.

3.1. Notations and preliminaries

We use plain letters (e.g., a , A) to denote scalars, bold lowercase letters (e.g., $\mathbf{a}$) to represent vectors, bold uppercase letters (e.g., $\mathbf{A}$) to represent matrices, bold calligraphic letters (e.g., $\mathcal{A}$) to denote tensors, and calligraphic letters (e.g., $\mathcal{A}$) to denote sets. A tensor is a generalization of a matrix to a multi-way data array. A N -order tensor is a multi-way data array $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, where I_n is the size of mode n . The $(i_1, i_2, \dots, i_N)$ -th element of $\mathcal{A}$ is denoted as $a_{i_1 i_2 \dots i_N}$. $\mathbf{a}^\top$ is the transpose of $\mathbf{a}$. The cardinality of set $\mathcal{A}$ is denoted as $|\mathcal{A}|$. The Kronecker delta is denoted by δ_{ij} and defined as

$$\delta_{ij} = \begin{cases} 1, & \text{if } i = j, \\ 0, & \text{otherwise.} \end{cases}$$

Definition 1. The mode- n unfolding (or matricization) of a tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, denoted by $\mathcal{A}_{(n)}$, is the process of rearranging the elements of $\mathcal{A}$ into a matrix $\mathbf{A}_{(n)} \in \mathbb{R}^{I_n \times (I_1 \dots I_{n-1} I_{n+1} \dots I_N)}$. The tensor element $(i_1, i_2, \dots, i_N)$ maps to the matrix element (i_n, j) , where

$$j = 1 + \sum_{\substack{k=1 \\ k \neq n}}^N (i_k - 1) \prod_{\substack{m=1 \\ m \neq n}}^{k-1} I_m. \quad (1)$$

This mapping ensures that each column of $\mathbf{A}_{(n)}$ corresponds to a mode- n fiber of $\mathcal{A}_{(n)}$, preserving the relative ordering of elements across all modes (Kolda & Bader, 2009).

The unfolding operation is introduced to enable the application of matrix-based linear algebra tools, such as the singular value decomposition, to high-order tensors. Although convolutional weights are naturally represented as tensors, many efficient decomposition techniques operate on matrices. By matricizing the tensor along a specific mode, we obtain an equivalent matrix representation that preserves all information of the original tensor without loss of generality (Kolda & Bader, 2009). This transformation allows us to leverage well-established matrix factorization methods while maintaining a direct correspondence with the original tensor through reshaping operations. In particular, in this work, we employ mode-0 unfolding to transform the convolutional weight tensor into a matrix where each row corresponds to a vectorized filter. This representation is especially suitable for our framework, as it enables us to interpret filter sharing as a low-rank approximation problem, where the shared basis filters and corresponding coefficients are naturally derived via truncated SVD. We deliberately choose mode-0 (output-channel) unfolding rather than unfolding along the input-channel or spatial modes, since it is the only choice that aligns each row

of the resulting matrix with one complete filter, the atomic unit that basis sharing is meant to operate on; unfolding along any other mode would instead mix entries from different filters within a single row, so that the resulting bases and coefficients would no longer correspond to a meaningful notion of a shared filter basis.

Definition 2. Let $\mathcal{A}, \mathcal{B} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ be tensors of the same size. The Frobenius inner product is defined as

$$\langle \mathcal{A}, \mathcal{B} \rangle_F = \sum_{i_1=1}^{I_1} \sum_{i_2=1}^{I_2} \dots \sum_{i_N=1}^{I_N} a_{i_1 \dots i_N} b_{i_1 \dots i_N}. \quad (2)$$

The Frobenius norm of a tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ is then defined as the square root of the inner product of $\mathcal{A}$ with itself:

$$\|\mathcal{A}\|_F = \sqrt{\langle \mathcal{A}, \mathcal{A} \rangle_F} = \sqrt{\sum_{i_1=1}^{I_1} \sum_{i_2=1}^{I_2} \dots \sum_{i_N=1}^{I_N} a_{i_1 \dots i_N}^2}.$$

This definition generalizes the familiar Frobenius norm for matrices (Kolda & Bader, 2009).

Definition 3. The singular value decomposition (SVD) of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a factorization of the form

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T = \sum_{r=1}^R \sigma_r \mathbf{u}_r \mathbf{v}_r^T, \quad (3)$$

where $R \leq \min(m, n)$ is the rank of $\mathbf{A}$, $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is a diagonal matrix whose diagonal entries are non-negative and represent the singular values, $\mathbf{U} \in \mathbb{R}^{m \times m}$ is an orthogonal matrix, containing the left singular values of $\mathbf{A}$, and $\mathbf{V} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix, containing the right singular values of $\mathbf{A}$. The singular values adopt the following convention $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_R \geq 0 = \sigma_{R+1} = \dots = \sigma_{\min(m, n)}$.

In our framework, the orthogonality of the right singular vectors $\{\mathbf{v}_r\}$ plays a central role. Specifically, these vectors are later reshaped into basis filters, and their orthogonality ensures that the resulting basis forms an orthonormal set under the Frobenius inner product. This property is crucial for our theoretical analysis, as it leads to a distance preservation result (Lemma 1), which establishes that distances between filters are exactly preserved in the coefficient space. We specifically require an orthonormal basis, rather than an unconstrained or non-orthogonal factorization such as a plain least-squares basis, because orthonormality is precisely what makes redundancy analysis in the coefficient space valid: without it, two coefficient vectors could appear similar under the Euclidean metric while the corresponding reconstructed filters are far apart (or vice versa), which would undermine the correctness of any pruning decision made directly on the coefficients. The truncated SVD is therefore not an arbitrary decomposition choice but the specific mechanism that provides this guarantee at essentially no additional computational cost.

Let us consider a convolutional neural network (CNN) composed of L convolutional layers. The l -th layer ($1 \leq l \leq L$) contains a weight tensor $\mathcal{W}^{(l)} \in \mathbb{R}^{O^{(l)} \times I^{(l)} \times K_h^{(l)} \times K_w^{(l)}}$, where $O^{(l)}$, $I^{(l)}$, $K_h^{(l)}$, and $K_w^{(l)}$ denote the number of output channels, input channels, and the kernel height and width, respectively. The tensor $\mathcal{W}^{(l)}$ consists of $O^{(l)}$ 3-order filters $\{\mathcal{W}_1^{(l)}, \mathcal{W}_2^{(l)}, \dots, \mathcal{W}_{O^{(l)}}^{(l)}\}$, where each filter $\mathcal{W}_o^{(l)} = \mathcal{W}_{o, \dots, \dots}^{(l)} \in \mathbb{R}^{I^{(l)} \times K_h^{(l)} \times K_w^{(l)}}$. Each filter is therefore naturally represented as a 3-order tensor corresponding to the input-channel and spatial kernel dimensions; this structure follows directly from the standard definition of convolutional layers and does not introduce any additional modeling assumptions. It also enables the proposed basis-sharing strategy to capture correlations jointly across channel and spatial modes. For brevity, the layer index (l) will be omitted when not essential.

Each convolutional layer maps an input tensor $\mathcal{I} \in \mathbb{R}^{I \times H_{in} \times W_{in}}$ to an output tensor $\mathcal{O} \in \mathbb{R}^{O \times H_{out} \times W_{out}}$, where H_{in} , W_{in} , H_{out} , and W_{out} are the

spatial dimensions of the input and output, respectively. The output spatial dimensions are given by the standard convolution formula:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p_h - d_h(K_h - 1) - 1}{s_h} + 1 \right\rfloor, \\ W_{out} = \left\lfloor \frac{W_{in} + 2p_w - d_w(K_w - 1) - 1}{s_w} + 1 \right\rfloor.$$

For simplicity, the bias term and batch dimension are omitted. The convolution operation can be expressed as

$$\mathcal{O}_o(h_{out}, w_{out}) = \sum_{i=1}^I \sum_{k_h=1}^{K_h} \sum_{k_w=1}^{K_w} \mathcal{I}(i, h_{in}, w_{in}) \mathcal{W}_o(i, k_h, k_w), \quad (4)$$

for $1 \leq o \leq O$, $1 \leq h_{out} \leq H_{out}$, and $1 \leq w_{out} \leq W_{out}$. Here, the input spatial indices (h_{in}, w_{in}) are functions of the output indices and kernel offsets, defined as

$$h_{in} = s_h h_{out} - p_h + d_h(k_h - 1), \\ w_{in} = s_w w_{out} - p_w + d_w(k_w - 1),$$

where (p_h, p_w) , (d_h, d_w) , and (s_h, s_w) denote the padding, dilation, and stride along the height and width dimensions, respectively. If (h_{in}, w_{in}) fall outside the valid input range, the corresponding input values are defined according to the padding scheme (e.g., zero-padding).

3.2. Basis sharing

We aim to approximate each 3-order filter $\mathcal{W}_o$, for $1 \leq o \leq O$, as a linear combination of B shared 3-order basis filters $\{\mathcal{B}_b\}_{b=1}^B$, with $\mathcal{B}_b \in \mathbb{R}^{I \times K_h \times K_w}$ and $B < O$. This approximation can be written as:

$$\mathcal{W}_o = \sum_{b=1}^B \mathbf{C}_{ob} \mathcal{B}_b, \quad (5)$$

where $\mathbf{C} \in \mathbb{R}^{O \times B}$ is the coefficient matrix, and $\mathbf{C}_{ob}$ is the weight of the b -th basis filter in reconstructing the o -th filter. Here, each coefficient $\mathbf{C}_{ob}$ is a scalar weight, and since all basis filters $\mathcal{B}_b$ share the same dimensions $I \times K_h \times K_w$, Eq. (5) represents a scalar-tensor linear combination. Therefore, the reconstructed filter $\mathcal{W}_o$ has dimensions $\mathbb{R}^{I \times K_h \times K_w}$.

By substituting Eq. (5) into Eq. (4), we obtain a new basis-sharing-based approach to compute the convolution. This approach involves a sequence of mappings using the basis filters and the coefficient matrix. The resulting equation for this basis-sharing-based convolution is:

$$\mathcal{O}_o(h_{out}, w_{out}) = \sum_{k_h=1}^{K_h} \sum_{k_w=1}^{K_w} \sum_{i=1}^I \mathcal{I}(i, h_{in}, w_{in}) \\ \cdot \sum_{b=1}^B \mathbf{C}_{ob} \mathcal{B}_b(i, k_h, k_w). \quad (6)$$

Starting from Eq. (6), we observe that the basis sharing-based convolution involves element-wise multiplications between the input tensor $\mathcal{I}$ and the basis filters $\mathcal{B}_b$ and the coefficient matrix $\mathbf{C}$. It is important to note that the order of the multiplication can be rearranged without affecting the final result. This flexibility allows the computation to be expressed as a convolution of the input tensor $\mathcal{I}$ with the B shared basis filters $\mathcal{B}_b$ along the input channel dimension, followed by a linear recombination using the coefficient matrix $\mathbf{C}$. Concretely, we first convolve $\mathcal{I}$ with each basis filter $\mathcal{B}_b$ to obtain an intermediate tensor $\mathcal{X}_b$. The collection of these B intermediate feature maps is then linearly combined according to $\mathbf{C}$ to produce the final output tensor. The following set of equations concisely captures these operations:

$$\mathcal{X}_b(h_{out}, w_{out}) = \sum_{k_h=1}^{K_h} \sum_{k_w=1}^{K_w} \sum_{i=1}^I \mathcal{I}(i, h_{in}, w_{in}) \mathcal{B}_b(i, k_h, k_w), \quad (7)$$

$$\mathcal{O}_o(h_{out}, w_{out}) = \sum_{b=1}^B \mathbf{C}_{ob} \mathcal{X}_b(h_{out}, w_{out}), \quad (8)$$

where $\mathcal{X} \in \mathbb{R}^{B \times H_{\text{out}} \times W_{\text{out}}}$.

The number of basis filters B determines the compression ratio of the proposed weight sharing scheme. The number of parameters of the original convolution layer is $OIK_h K_w$. For the basis filters, the number of parameters is $B I K_h K_w$ and the size of the coefficient matrix $\mathbf{C}$ is BO , resulting in a total number of parameters of $B(IK_h K_w + O)$. The reduction in computational complexity arises because the original convolution with O filters is replaced by two consecutive operations: a convolution with only B shared basis filters followed by a lightweight linear recombination step using the coefficient matrix. The computational complexity of the original layer is $O(IK_h K_w HW)$, assuming $H_{\text{in}} = H_{\text{out}} = H$, and $W_{\text{in}} = W_{\text{out}} = W$ for simplicity. For the proposed weight sharing scheme, the computational complexity is $O(B(IK_h K_w + O)HW)$. To ensure a non-negative reduction in both the number of parameters and computational complexity, the following constraint must be satisfied:

$$B \leq \frac{OIK_h K_w}{IK_h K_w + O}. \quad (9)$$

The objective is to find the optimal basis filters and corresponding coefficients that minimize the reconstruction error:

$$\min_{\{\mathcal{B}_b\}} \sum_{o=1}^O \left\| \mathcal{W}_o - \sum_{b=1}^B \mathbf{C}_{ob} \mathcal{B}_b \right\|_F^2, \quad (10)$$

where $\|\cdot\|_F$ denotes the Frobenius norm. This optimization problem can be solved via multiple approaches. A common method is alternating minimization, where we iteratively update the coefficient matrix $\mathbf{C}$ (while keeping the basis filters fixed) and then update the basis filters $\{\mathcal{B}_b\}$ (while keeping $\mathbf{C}$ fixed). This method directly minimizes the reconstruction error and can incorporate regularizations, but it is computationally expensive, prone to local minima, and sensitive to initialization. Another approach involves clustering-based techniques, such as K-means, where the basis filters are chosen as cluster centroids of the original filters and coefficients are assigned accordingly. While clustering is simple, fast, and easy to implement, it typically suffers from lower approximation quality and limited flexibility in modeling fine-grained correlations between filters.

In this work, we adopt the mode-0 truncated SVD as a principled and efficient approach for basis sharing. Specifically, the 4-order convolutional weight tensor $\mathcal{W} \in \mathbb{R}^{O \times I \times K_h \times K_w}$ is first unfolded along the output channel mode to obtain a matrix $\mathcal{W}_{(0)} \in \mathbb{R}^{O \times (IK_h K_w)}$. Applying the SVD in Eq. (3) yields:

$$\mathcal{W}_{(0)} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T = \sum_{r=1}^{\min(O, IK_h K_w)} \sigma_r \mathbf{U}_{:,r} \mathbf{V}_{r,:}^T, \quad (11)$$

where $\mathbf{U} \in \mathbb{R}^{O \times O}$ and $\mathbf{V} \in \mathbb{R}^{IK_h K_w \times IK_h K_w}$ contain the left and right singular vectors, respectively, and $\mathbf{\Sigma} \in \mathbb{R}^{O \times IK_h K_w}$ is a diagonal matrix of singular values σ_r , for $1 \leq r \leq \min(O, IK_h K_w)$. To obtain the basis-sharing mechanism described in Eq. (5), we retain only the top B singular components of the decomposition:

$$\mathcal{W}_{(0)}^{(B)} = \mathbf{U}_{[:,1:B]} \mathbf{\Sigma}_{1:B,1:B} \mathbf{V}_{[1:B,:]}^T. \quad (12)$$

The truncated factors naturally yield the coefficient matrix and basis filters as:

$$\mathbf{C} = \mathbf{U}_{[:,1:B]} \mathbf{\Sigma}_{1:B,1:B}, \quad (13)$$

$$\mathcal{B}_b = \text{reshape}(\mathbf{V}_{[b,:]} [I, K_h, K_w]), \quad b = 1, \dots, B. \quad (14)$$

Eq. (13) defines a compact coefficient matrix $\mathbf{C} \in \mathbb{R}^{O \times B}$, while Eq. (14) constructs each shared basis filter $\mathcal{B}_b$ by reshaping the corresponding column vector of $\mathbf{V}$ into a $[I \times K_h \times K_w]$ tensor.

The mode-0 SVD offers a closed-form, globally optimal rank- B approximation in Frobenius norm as guaranteed by the Eckart–Young–Mirsky theorem (Eckart & Young, 1936), making it both principled and efficient. Specifically, the associated reconstruction error is

bounded by the sum of the squared discarded singular values:

$$\left\| \mathcal{W}_{(0)} - \mathcal{W}_{(0)}^{(B)} \right\|_F^2 = \sum_{i=B+1}^{\min(O, IK_h K_w)} \sigma_i^2. \quad (15)$$

This bound guarantees that mode-0 SVD yields the optimal rank- B approximation of the unfolded weight tensor $\mathcal{W}_{(0)}$ in the Frobenius norm. Consequently, it provides a closed-form solution to a relaxed version of the basis sharing optimization problem (10), formulated over the mode-0 unfolding.

From a computational perspective, the truncated SVD used for basis construction admits a well-established complexity characterization. For a matrix $\mathcal{W}_{(0)} \in \mathbb{R}^{O \times (IK_h K_w)}$, computing a rank- B truncated SVD typically requires $O(IK_h K_w B)$ operations using modern partial SVD algorithms (Halko et al., 2011). This complexity scales linearly with the retained rank B and is lower than that of full SVD, which scales as $O(IK_h K_w \min(O, IK_h K_w))$. Moreover, the decomposition is performed only once per layer and does not involve iterative optimization procedures such as alternating minimization or search-based basis construction strategies. As a representative example, consider a large convolutional layer from a ResNet-50 bottleneck block with 1024 output channels, 1024 input channels, and kernel size 3×3 . After unfolding, this corresponds to a matrix of size 1024×9216 . In our implementation, computing the truncated SVD for this matrix requires approximately 5497 ± 81 milliseconds, depending on the numerical backend and computational environment. It is worth noting that the truncated SVD is applied only during the compression stage prior to pruning and fine-tuning, so it does not affect inference-time efficiency.

An important advantage of the formulation proposed in Eqs. (13) and (14) lies in the orthonormality of the basis. First, previous works have shown that orthogonal weights facilitate the optimization of deep networks by mitigating gradient explosion and vanishing issues (Hu et al., 2020). Second, the orthonormality of the right singular vectors ensures that Euclidean distances in the coefficient space directly reflect distances between the corresponding filters. This property is particularly valuable for redundancy analysis and pruning, as will be discussed in Subsection 3.3.

3.3. Coefficient pruning

The basis-sharing approach decomposes filters into a combination of basis filters. The coefficient matrix encodes how each filter is formed from the shared bases. The basis sharing method reduces computational costs and the number of trainable parameters. However, it does not fully exploit the redundancies inherent in the models. The redundancies exist potentially in the coefficient matrix. In the following, we first show that if two filters are similar, their corresponding coefficient vectors should be similar. We then empirically show that there is redundancy among the coefficient matrix's rows.

Let us consider two filters, $\mathcal{W}_m$ and $\mathcal{W}_n$, following Eq. (5), their filter basis approximations are expressed as $\mathcal{W}_m = \sum_{b=1}^B \mathbf{C}_{mb} \mathcal{B}_b$ and $\mathcal{W}_n = \sum_{b=1}^B \mathbf{C}_{nb} \mathcal{B}_b$. The distance between two filters, measured via their Frobenius norm distance, is expressed as

$$\left\| \mathcal{W}_m - \mathcal{W}_n \right\|_F^2 = \left\| \sum_{b=1}^B (\mathbf{C}_{mb} - \mathbf{C}_{nb}) \mathcal{B}_b \right\|_F^2. \quad (16)$$

Lemma 1 (Distance Preservation under Orthonormal Basis). *Let $\{\mathcal{B}_b\}_{b=1}^B$ be an orthonormal set of basis filters, i.e., $\langle \mathcal{B}_i, \mathcal{B}_j \rangle_F = \delta_{ij}$. Then, for any two filters $\mathcal{W}_m$ and $\mathcal{W}_n$ expressed in this basis, their Frobenius distance equals the squared Euclidean distance between their coefficient vectors:*

$$\left\| \mathcal{W}_m - \mathcal{W}_n \right\|_F^2 = \left\| \mathbf{C}_m - \mathbf{C}_n \right\|_2^2. \quad (17)$$

Proof. Let us define $\Delta_b = \mathbf{C}_{mb} - \mathbf{C}_{nb}$. Then,

$$\mathcal{W}_m - \mathcal{W}_n = \sum_{b=1}^B \Delta_b \mathcal{B}_b. \quad (18)$$

Using the definition of the Frobenius inner product (Definition 2),

$$\|\mathcal{W}_m - \mathcal{W}_n\|_F^2 = \left\langle \sum_{b=1}^B \Delta_b \mathcal{B}_b, \sum_{b'=1}^B \Delta_{b'} \mathcal{B}_{b'} \right\rangle_F \quad (19)$$

Expanding the inner product gives

$$\left\langle \sum_{b=1}^B \Delta_b \mathcal{B}_b, \sum_{b'=1}^B \Delta_{b'} \mathcal{B}_{b'} \right\rangle_F = \sum_{b=1}^B \sum_{b'=1}^B \Delta_b \Delta_{b'} \langle \mathcal{B}_b, \mathcal{B}_{b'} \rangle_F. \quad (20)$$

By orthonormality, $\langle \mathcal{B}_b, \mathcal{B}_{b'} \rangle_F = \delta_{bb'}$, hence all cross terms vanish, yielding

$$\|\mathcal{W}_m - \mathcal{W}_n\|_F^2 = \sum_{b=1}^B \Delta_b^2 = \|\mathbf{C}_m - \mathbf{C}_n\|_2^2. \quad (21)$$

□

In our framework, the orthonormality condition naturally holds. Specifically, the basis filters $\mathcal{B}_b$ are obtained by reshaping the right singular vectors $\mathbf{V}_{:,b} \in \mathbb{R}^{I K_h K_w}$ from the SVD (Eq. (14)). Since these vectors satisfy $\mathbf{V}_{:,b}^\top \mathbf{V}_{:,b'} = \delta_{bb'}$, and because the Frobenius norm is invariant under reshaping (i.e., $\|\mathcal{A}\|_F = \|\text{vec}(\mathcal{A})\|_2$), the resulting basis filters are orthonormal with respect to the Frobenius inner product.

Lemma 1 establishes that, under this orthonormal basis, the Frobenius distance between filters is exactly preserved in the coefficient space. This result has an important implication: similarity relationships between filters are faithfully captured by the Euclidean geometry of their coefficient vectors. Consequently, redundancy detection and comparison can be performed efficiently in the low-dimensional coefficient space, without loss of fidelity.

In particular, if two filters are similar in the original space, their coefficient vectors will also be close in Euclidean distance. This enables structured pruning to be carried out directly on the rows of the coefficient matrix, effectively reducing the number of output channels while preserving the essential geometric structure of the original filters.

We define a similarity function between two coefficient rows $\mathbf{C}_m$ and $\mathbf{C}_n$ using a Gaussian kernel: $\kappa : \mathbb{R}^B \times \mathbb{R}^B \rightarrow [0, 1]$, expressed as

$$\kappa(\mathbf{C}_m, \mathbf{C}_n) = \exp\left(-\frac{\|\mathbf{C}_m - \mathbf{C}_n\|_F^2}{2\sigma^2}\right), \quad (22)$$

where the kernel bandwidth σ controls the sensitivity of the similarity to pairwise distances. We define the affinity (i.e., square, symmetric) matrix $\mathbf{A} \in \mathbb{R}^{O \times O}$, where

$$\mathbf{A}_{mn} = \kappa(\mathbf{C}_m, \mathbf{C}_n), \quad (23)$$

as the similarity matrix. An inappropriate choice of σ may lead to overly uniform similarity scores ($\mathbf{A}_{mn} \approx 1$) or vanishingly small values ($\mathbf{A}_{mn} \approx 0$), both of which reduce the effectiveness of the diversity-based pruning. To balance sensitivity and robustness, we empirically select σ as the median of the pairwise distances:

$$\sigma = \text{median}_{1 \leq m < n \leq O} \|\mathbf{C}_m - \mathbf{C}_n\|_F. \quad (24)$$

This choice is robust to outliers and ensures that the similarity matrix $\mathbf{A}$ spans a wide dynamic range. Specifically, since the Gaussian kernel is a monotonic function of the squared Euclidean distance, and σ is chosen as the median distance, the kernel argument $\frac{\|\mathbf{C}_m - \mathbf{C}_n\|_F^2}{2\sigma^2}$ will be less than or equal to 1 for approximately half of the off-diagonal entries. Consequently, about half of the similarity values will satisfy $\mathbf{A}_{mn} \geq \exp(-\frac{1}{2}) \approx 0.6065$, and all $\mathbf{A}_{mn}$ lie in the interval $(0, 1]$. This guarantees that the similarity scores retain meaningful variation, enhancing the contrast between redundant and diverse rows, which is crucial for the pruning process. Fig. 3 visualizes the affinity matrices $\mathbf{A}$ for coefficient vectors across different layers, architectures, and datasets. Dark red blocks along the diagonal indicate groups of highly similar filters, revealing redundancy that can be exploited by pruning. In ResNet-56, moderate clusters suggest partial redundancy, whereas VGG-16 exhibits more diverse filters with fewer pronounced clusters. WideResNet-50

shows a higher density of sombre entries, reflecting strong redundancy. These visualizations confirm that the Gaussian-kernel similarity with median-based σ effectively spreads the affinity values across a meaningful range, facilitating contrast between redundant and diverse filters, which is crucial for our coefficient-pruning strategy. A more detailed empirical ablation study on the choice of the kernel bandwidth σ is provided in Subsection 5.4.

To remove redundancy in the coefficient matrix while preserving filter diversity, we consider the following optimization problem: given the affinity matrix $\mathbf{A} \in \mathbb{R}^{O \times O}$, we aim to find a subset $\mathcal{P} \subset \{1, \dots, O\}$ with $|\mathcal{P}| = P$, where P is the number of the rows to be preserved in the coefficient matrix, such that the maximum intra-set similarity is minimized:

$$\mathcal{P}^* = \arg \min_{\substack{\mathcal{P} \subset \{1, \dots, O\} \\ |\mathcal{P}| = P}} \max_{\substack{m, n \in \mathcal{P} \\ m \neq n}} \mathbf{A}_{mn}. \quad (25)$$

This formulation seeks to retain the subset of coefficient rows such that the least diverse pair among them is as dissimilar as possible. Since the above combinatorial problem is NP-hard, we adopt a farthest-first traversal strategy (Algorithm 1) which constructs $\mathcal{P}$ by iteratively selecting the row with the lowest maximum similarity to the already selected subset. The first element is chosen via a redundancy-aware initialization that minimizes the total similarity to all other rows, ensuring the anchor filter is globally distinct.

Algorithm 1 Redundancy-Aware Coefficient Selection.

Require: Affinity matrix $\mathbf{A} \in \mathbb{R}^{O \times O}$, target number of coefficients to be preserved P

Ensure: Index set $\mathcal{P} \subset \{1, \dots, O\}$ of size P

- 1: Initialize $\mathcal{R} \leftarrow \{1, \dots, O\}$
 - 2: Select $m_0 \leftarrow \arg \min_m \sum_{n \neq m} \mathbf{A}_{mn}$ ▷ Index of the least redundant coefficient
 - 3: $\mathcal{P} \leftarrow \{m_0\}$, $\mathcal{R} \leftarrow \mathcal{R} \setminus \{m_0\}$
 - 4: **while** $|\mathcal{P}| < P$ **do**
 - 5: **for** $m \in \mathcal{R}$ **do**
 - 6: $p_m \leftarrow \max_{n \in \mathcal{P}} \mathbf{A}_{mn}$
 - 7: **end for**
 - 8: $m^* \leftarrow \arg \min_{m \in \mathcal{R}} p_m$
 - 9: $\mathcal{P} \leftarrow \mathcal{P} \cup \{m^*\}$, $\mathcal{R} \leftarrow \mathcal{R} \setminus \{m^*\}$
 - 10: **end while**
 - 11: Return $\mathcal{P}$
-

3.4. Complexity analysis of the compressed model

The complexity of the compressed convolutional layer is jointly determined by the number of basis filters B , used for weight sharing, and the number of preserved coefficients P , retained after pruning. The original convolutional layer contains $OIK_h K_w$ parameters. When basis sharing is applied, the parameters are decomposed into B basis filters and a coefficient matrix $\mathbf{C}$. After pruning, the total number of parameters becomes $BK_h K_w + BP$. Hence, the parameter reduction ratio (RR) is given by

$$\text{RR}_{\text{params}} = 1 - \frac{BK_h K_w + BP}{OIK_h K_w}. \quad (26)$$

Eq. (26) provides an intuitive measure of how much the layer size has been compressed relative to the original model. Regarding computational cost, the number of multiply-accumulate operations (MACs) in the original convolutional layer is $OIK_h K_w HW$, assuming for simplicity that $H_{\text{in}} = H_{\text{out}} = H$ and $W_{\text{in}} = W_{\text{out}} = W$. In the compressed layer, convolution is first performed with the B basis filters, producing intermediate feature maps, and then linearly recombined using the P preserved coefficients. The total cost therefore becomes $BK_h K_w HW + BPHW$, leading to the MACs reduction ratio:

$$\text{RR}_{\text{MACs}} = 1 - \frac{BK_h K_w HW + BPHW}{OIK_h K_w HW}. \quad (27)$$

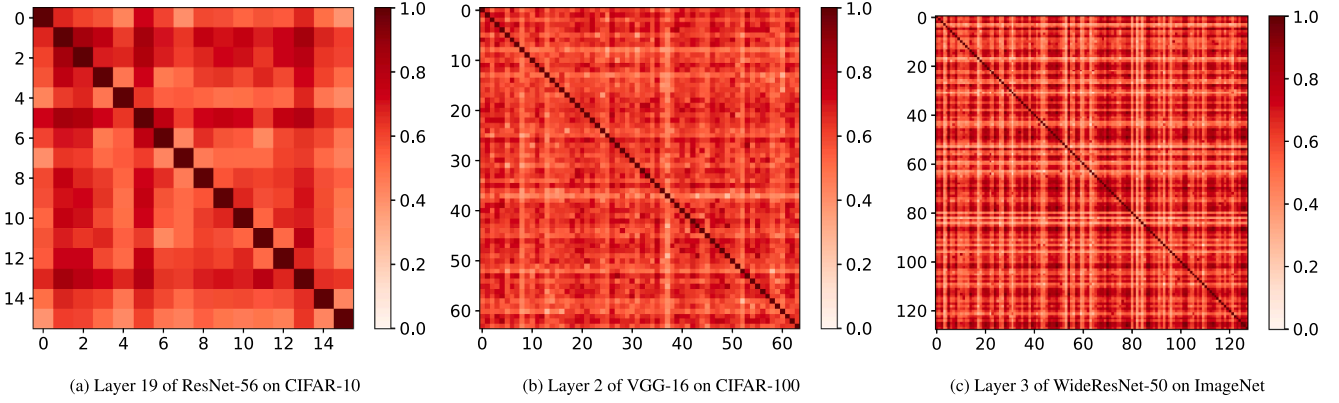


Fig. 3. Affinity matrices computed from coefficient vectors across different architectures and datasets, illustrating redundancy patterns among coefficients. Darker red blocks indicate stronger similarity (higher redundancy), while brighter regions correspond to more diverse coefficients.

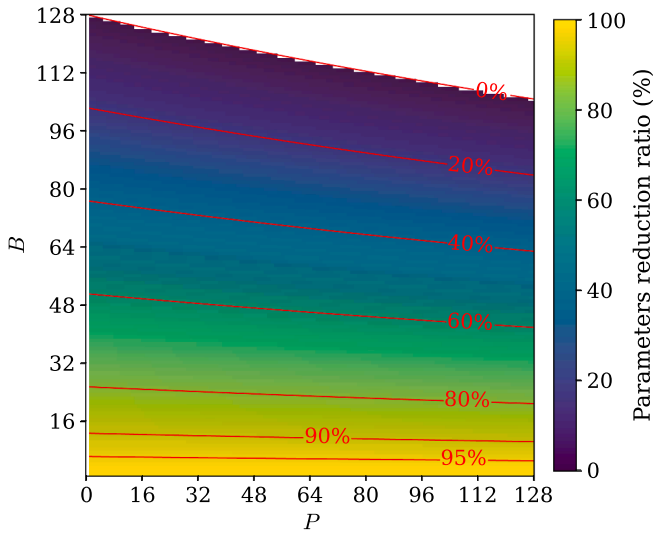


Fig. 4. Heatmap showing the parameter reduction ratio as a function of B and P . Contour lines indicate levels of constant reduction. The reduction ratio increases as either B or P decreases, with a stronger influence from B .

Eq. (27) quantifies the corresponding computational savings. Both Eqs. (26) and (27) show that the compression benefits depend jointly on B and P . A smaller B directly reduces both storage and computation, while pruning coefficients (lower P) further decreases the size of the coefficient matrix and the recombination cost.

To better illustrate how the number of basis filters (B) and preserved coefficients (P) jointly affect model compactness, we analyze the parameter reduction ratio defined in Eq. (26) using the third convolutional layer of VGG-16, where $O = 128$, $I = 64$, and $K_h = K_w = 3$. Fig. 4 presents a heatmap depicting the combined effect of (B, P) on the parameter reduction ratio. Each contour line corresponds to a constant level of reduction, highlighting regions that yield similar compression benefits. The figure clearly shows that the reduction ratio increases as either B or P decreases, but the effect is nonlinear: reducing B has a stronger impact than pruning coefficients alone. This confirms that most of the computational and storage savings come from limiting the number of basis filters, whereas coefficient pruning offers finer-grained control.

To further analyze this behavior, Fig. 5 plots the reduction ratio with respect to P for several fixed values of B . When B is small (e.g., $B \leq 16$), even large values of P maintain high compression levels, suggesting that the layer's expressiveness can be largely retained with only a few basis filters. Conversely, when B increases, the compression rapidly diminishes, emphasizing the necessity of a careful balance between the num-

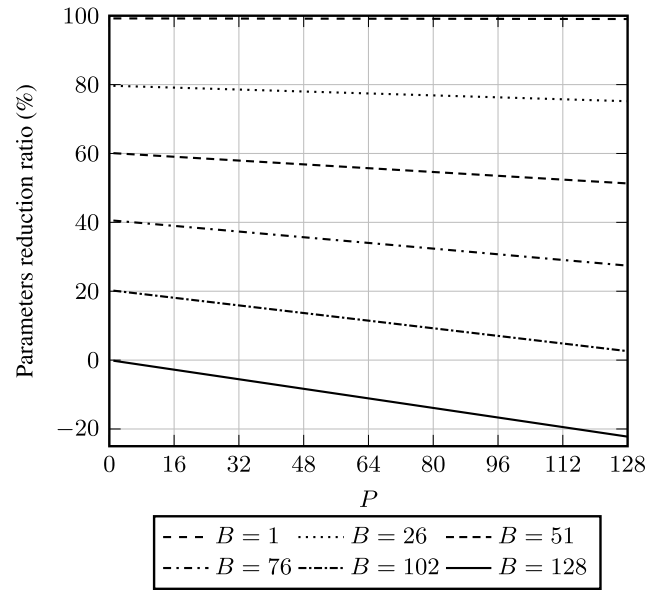


Fig. 5. Reduction ratio as a function of P for different values of B . Smaller B or lower P yields higher compression, with B exerting a stronger influence on the overall reduction.

ber of basis filters and the number of preserved coefficients. Together, these visualizations provide an intuitive understanding of how the reduction ratio interacts with B and P , supporting the selection strategy for determining these hyperparameters.

It is worth noting that Eqs. (26) and (27) describe the case of a single layer. In practice, when extending to the full network with L layers, the number of preserved coefficients in layer l also determines the input dimensionality of the next layer, i.e., $P^{(l)} = I^{(l+1)}$. Removing a row in the coefficient matrix thus eliminates the corresponding output feature map and, consequently, the associated input channel in the following layer. Therefore, the choice of $P^{(l)}$ impacts both the l -th and $(l+1)$ layers. The overall parameter reduction ratio of the entire network is expressed as

$$RR_{\text{params}} = 1 - \frac{\sum_{l=1}^L (B^{(l)} I^{(l)} K_h^{(l)} K_w^{(l)} + B^{(l)} P^{(l)})}{\sum_{l=1}^L O^{(l)} I^{(l)} K_h^{(l)} K_w^{(l)}}, \quad (28)$$

where $P^{(0)} = I^{(1)}$. The MACs reduction ratio can be derived analogously:

$$RR_{\text{MACs}} = 1 - \frac{\sum_{l=1}^L (B^{(l)} I^{(l)} K_h^{(l)} K_w^{(l)} + B^{(l)} P^{(l)}) H^{(l)} W^{(l)}}{\sum_{l=1}^L O^{(l)} I^{(l)} K_h^{(l)} K_w^{(l)} H^{(l)} W^{(l)}}, \quad (29)$$

These theoretical reductions are further validated empirically through measured parameter and MAC reductions across multiple architectures in Section 4, as well as real latency benchmarks on graphics processing unit (GPU), central processing unit (CPU), and embedded platforms in Subsection 5.6, demonstrating that the proposed compression strategy leads to consistent improvements not only in model compactness but also in practical inference efficiency. A detailed ablation study analyzing the interplay between $B^{(l)}$ and $P^{(l)}$ across layers is presented in Subsection 5.3.

3.5. Overall algorithm

The proposed SaP framework integrates basis sharing and coefficient pruning into a unified model compression pipeline, aiming to eliminate redundancy in both the filter and feature domains while maintaining high representational fidelity. Algorithm 2 summarizes the complete procedure. Given a pretrained convolutional network with L layers, SaP proceeds in two main stages: basis sharing and coefficient pruning, followed by fine-tuning to restore accuracy.

Algorithm 2 SaP: Basis Sharing and Coefficient Pruning.

Require: Pretrained model with the weights $\{\mathcal{W}^{(l)}\}_{l=1}^L$, number of basis filters $\{B^{(l)}\}_{l=1}^L$, number of coefficients to be preserved $\{P^{(l)}\}_{l=1}^L$

- 1: **for** $l = 1$ to L **do**
- 2: Perform Equation (12) ▷ Basis sharing
- 3: Get $C^{(l)}$ from Equation (13)
- 4: Get $B^{(l)}$ from Equation (14)
- 5: Form $A^{(l)}$ from Equation (23) ▷ Coefficient pruning
- 6: Identify indices of coefficients to be preserved P^l from $A^{(l)}$ and $P^{(l)}$ using Algorithm 1.
- 7: Keep $C_{P^l}^{(l)}$, prune $C_{\bar{P}^l}^{(l)}$.
- 8: **end for**
- 9: Freeze $\{B^{(l)}\}_{l=1}^L$
- 10: Fine-tune the remaining coefficients $\{C_{P^l}^{(l)}\}_{l=1}^L$

Ensure: Compressed model

In the first stage, basis sharing is performed independently for each convolutional layer. For a layer l , the weight tensor $\mathcal{W}^{(l)}$ is unfolded along its output-channel mode and factorized via the mode-0 truncated SVD as in Eq. (12). From the truncated factors, the coefficient matrix $C^{(l)}$ and the shared basis filters $B^{(l)}$ are derived using Eqs. (13) and (14), respectively. This decomposition expresses all filters as linear combinations of $B^{(l)}$ shared bases, significantly reducing parameter redundancy.

In the second stage, coefficient pruning is applied directly in the coefficient space. For each layer, an affinity matrix $A^{(l)}$ is constructed according to Eq. (23) to measure the pairwise similarity between coefficient vectors. Based on $A^{(l)}$ and the target number of coefficients to preserve $P^{(l)}$, the indices of the most representative coefficients P^l are identified using the redundancy-aware coefficient selection strategy (Algorithm 1). Coefficients corresponding to redundant output channels are pruned, while the selected subset $C_{P^l}^{(l)}$ is retained.

After pruning, the basis filters $\{B^{(l)}\}_{l=1}^L$ are frozen, and the remaining coefficients $\{C_{P^l}^{(l)}\}_{l=1}^L$ are fine-tuned on the training dataset to compensate for potential information loss. Fine-tuning is performed using the specified optimizer, learning rate schedule, batch size, and regularization hyperparameters, as detailed in Subsection 4.1.

4. Experiments

4.1. Experimental settings

4.1.1. Architectures and datasets

We validate the proposed SaP framework across twelve architectures and four datasets to thoroughly assess its generality and scalability. The experiments cover both classical and modern convolutional backbones, transformer-based, and hybrid CNN-transformer architectures, spanning from compact to large-scale models.

For small- to medium-scale datasets, the evaluation includes VGG-16 (Simonyan & Zisserman, 2015), ResNet-20/32/56/110 (He et al., 2016), DenseNet-40 (Huang et al., 2017), GoogLeNet (Szegedy et al., 2015), and SqueezeNet (Iandola et al., 2016). These architectures represent diverse design philosophies such as plain convolutional networks (VGG), residual connections (ResNet), dense connectivity (DenseNet), multi-branch topologies (GoogLeNet), and parameter-efficient fire-module design (SqueezeNet), allowing us to examine the versatility of SaP across different structural configurations. Unlike the other architectures, which are evaluated in the main classification and detection comparisons of Section 4, SqueezeNet is used specifically in the compression-efficiency ablation study (Subsection 5.5). On large-scale datasets, the study further considers ResNet-18/34/50, MobileNetV2 with inverted residual blocks (Sandler et al., 2018), and EfficientNet-B0 with mobile inverted bottleneck blocks (Tan & Le, 2019), which together provide a representative coverage of deep, lightweight, and NAS-optimized models. This selection enables a systematic investigation of SaP across varying network capacities, depths, and computational complexities. To assess the applicability of SaP beyond convolutional architectures, we further extend our evaluation to transformer-based and hybrid models. Specifically, we consider DeiT (Touvron et al., 2021), a representative vision transformer architecture, as well as RT-DETR (Zhao et al., 2024), a modern hybrid CNN-transformer detector. For DeiT, SaP is applied to linear projection layers. For RT-DETR, SaP is applied to both the convolutional backbone and the linear projection layers in the transformer encoder and decoder. In addition, SaP is applied to the convolutional backbones of FasterRCNN (Ren et al., 2017), Mask-RCNN, and Keypoint-RCNN (He et al., 2020), enabling evaluation across object detection, instance segmentation, and human pose estimation tasks on the MS-COCO 2017 dataset (Lin et al., 2014).

The classification experiments are conducted on three standard benchmarks. CIFAR-10 and CIFAR-100 (Krizhevsky et al., 2009) contain 50,000 training and 10,000 test images of size 32×32 , distributed across 10 and 100 classes, respectively. ImageNet (ILSVRC-2012) (Russakovsky et al., 2015) comprises 1.28 million training and 50,000 validation images covering 1000 categories. For dense prediction tasks, we use the MS-COCO 2017 dataset (Lin et al., 2014), which includes 118,000 training and 5000 validation images annotated for detection, segmentation, and keypoint detection.

4.1.2. Evaluation protocols

The model is evaluated based on three metrics: accuracy, Multiply-Accumulate Operations (MACs), and number of parameters (Params). MACs and Params reflect computational and storage costs, respectively. The percentage reduction ratio (RR) measures the relative decrease in MACs or Params compared to the uncompressed model:

$$RR = \left(1 - \frac{\text{compressed}}{\text{uncompressed}}\right) \times 100\%. \quad (30)$$

Table 1
Fine-tuning hyperparameters.

Dataset	Model	Scheduler	Optimizer	Epochs	Learning rate	Batch size	Weight decay
CIFAR-10/100	All	CosineAnnealing	SGD	300	0.1	128	5×10^{-4}
ImageNet	ResNet-18/34/50	CosineAnnealing	SGD	120	0.05	128	1×10^{-4}
ImageNet	MobileNetV2	CosineAnnealing	SGD	90	0.05	128	4×10^{-5}
ImageNet	EfficientNet-B0	CosineAnnealing	SGD	90	0.05	128	2×10^{-5}
ImageNet	DeiT	CosineAnnealing	AdamW	120	0.001	512	2×10^{-4}
COCO	Faster/Mask-RCNN	MultiStep	SGD	26	0.02	16	1×10^{-4}
COCO	Keypoint-RCNN	MultiStep	SGD	46	0.02	16	1×10^{-4}
COCO	RT-DETR	MultiStep	AdamW	72	0.0001	4	1×10^{-4}

Concerning performance, top-1 and top-5 accuracies are reported for classification tasks. For object detection, instance segmentation, and human keypoint detection, we use the standard average precision (AP) metrics: $AP_{50:95}$, which is averaged over uniformly sampled IoU thresholds from 0.50 to 0.95 with a step size of 0.05, and AP_{50} , which corresponds to the AP at a single intersection over union (IoU) threshold of 0.50 and is widely used in real-world scenarios due to its simplicity and interpretability. The accuracy drop is computed as the difference in top-1 accuracy between the compressed model and the uncompressed baseline:

$$\Delta \text{accuracy} = \text{accuracy}_{\text{compressed}} - \text{accuracy}_{\text{uncompressed}}. \quad (31)$$

In all experiments, a good compression method is expected to achieve a high reduction ratio with a minimal accuracy drop, indicating efficient compression with limited impact on task performance.

4.1.3. Fine-tuning configuration

To support exact reproduction of our results, we report in Table 1, for every dataset–architecture combination, the optimizer, learning-rate scheduler, number of training epochs, initial learning rate, batch size, and weight decay used during fine-tuning. We adopt the training configurations from related works (Pham et al., 2025b, 2026; Zhang et al., 2025) to ensure fair and consistent evaluation. Stochastic gradient descent (SGD) with momentum is used for most models due to its strong generalization, while AdamW is employed for the transformer-based models (DeiT Touvron et al., 2021, and RT-DETR Zhao et al., 2024). CosineAnnealing scheduling is applied to classification tasks for smooth learning rate decay, whereas MultiStep scheduling is used for detection models. These configurations provide balanced convergence and regularization across datasets and architectures.

For completeness and to allow the exact input pipeline to be reproduced, we detail below the full preprocessing procedure applied to each dataset. All datasets are preprocessed following standard protocols used in prior works (Kumar et al., 2025; Pham et al., 2025b; Tofigh et al., 2026a; Zhang et al., 2025):

- **CIFAR-10/100.** Training images are randomly cropped to 32×32 pixels with 4-pixel padding and horizontally flipped, then converted to tensors and normalized using the standard mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2023, 0.1994, 0.2010). Test images are only converted to tensors and normalized.
- **ImageNet.** Training images are processed with random resized cropping, horizontal flipping, and normalization using the standard mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225]. Validation images are resized to 256 pixels on the shorter side, center-cropped to 224×224 , and normalized.
- **COCO.** Training images are resized to 640×640 pixels and augmented with horizontal flipping and copy-paste augmentation. Validation images are resized to the same resolution without augmentation.

All experiments were conducted in a single, fixed hardware and software environment, which we report here for full reproducibility: fine-tuning

was performed using full (FP32) precision, without mixed-precision (AMP) training, distributed across 4 GPUs via PyTorch’s DistributedDataParallel (DDP). NVIDIA Tesla V100 GPUs with Intel Xeon Gold 6240 CPUs, using PyTorch 2.9, Torchvision 0.24 (for data preprocessing), and CUDA 12.8. The latency measurements in Subsection 5.6 likewise use FP32 precision throughout, on each single target device, without any AMP-based acceleration.

In our framework, only the fine-tuning stage involves stochasticity arising from stochastic optimization and data shuffling, as the decomposition and pruning stages are deterministic. Since the variability introduced by fine-tuning is typically small, we follow the common practice in network compression studies and report results from a single representative run for each model, using random seed 0 for all main results reported in Section 4. For small-scale datasets such as CIFAR-10 and CIFAR-100, we found that different random seeds yield highly consistent outcomes (see Subsection 5.7, where we repeat fine-tuning with seeds {0, 1, 2, 3, 4} for CIFAR-scale models and {0, 1} for MobileNetV2/ImageNet). For large-scale datasets such as ImageNet, repeating the experiments is computationally expensive, but we later empirically observed that performance variation across seeds remains negligible.

4.1.4. Baselines for comparison

SaP is compared with 73 related works in weight sharing (S) (Chen et al., 2024a, 2023; Kang & Kim, 2021; Li et al., 2022, 2020a, 2019; Pham et al., 2026; Phan et al., 2024; Shin et al., 2024; Sun et al., 2021; Tu et al., 2025; Wong et al., 2024; Yang et al., 2020, 2019; Zhang et al., 2025), pruning (P) (Chang et al., 2023; Chen et al., 2024a, 2021, 2024b, 2022, 2024c; Dai et al., 2021; Fan et al., 2026; Ganjdanesh et al., 2024; Gao et al., 2026, 2024; Guan et al., 2023; Guo et al., 2024; Hou et al., 2024; Jayasimhan & Pabitha, 2024; Ji et al., 2025; Kumar et al., 2025; Lee & Lee, 2026; Lee & Song, 2024; Li et al., 2025, 2024, 2026; Lin et al., 2025, 2023, 2020a,b; Liu et al., 2024a, 2022, 2023, 2024b; Lu et al., 2023; Niu et al., 2023; Pham et al., 2023, 2024a, 2025a,b; Qian et al., 2024; Ravi Ganesh et al., 2024; Ruan et al., 2021; Shi et al., 2024; Shin et al., 2024; Su et al., 2023; Sui et al., 2025, 2021; Sun & Shi, 2024; Tang et al., 2025; Tofigh et al., 2026a,b; Tu et al., 2025; Wong et al., 2024; Wu et al., 2024; Yang et al., 2024; Yuan et al., 2024; Zhang & Liu, 2025; Zhang & Freris, 2024; Zhang et al., 2023, 2025), NAS (N) (Chen et al., 2024b; Ganjdanesh et al., 2024; Kumar et al., 2025; Lee & Song, 2024; Li et al., 2026; Lin et al., 2020b; Liu et al., 2024a; Lu et al., 2023; Su et al., 2023; Wu et al., 2024; Xiao et al., 2023; Zhang et al., 2025), knowledge distillation (K) (Hou et al., 2024; Lee & Song, 2024; Li et al., 2025; Lu et al., 2023; Su et al., 2023; Zhang & Liu, 2025), and low-rank decomposition (L) (Chen et al., 2023; Dai et al., 2021, 2025; Gabor & Zdunek, 2026; Guo et al., 2025; Hajimolahoseini et al., 2022; Lee & Lee, 2026; Liebenwein et al., 2021; Pham et al., 2023, 2024a, 2025a, 2026; Phan et al., 2024; Ruan et al., 2021; Sui et al., 2025; Sun et al., 2021; Xiao et al., 2023). In addition to these 73 cited related works, each of the eleven main-comparison architectures (i.e., excluding SqueezeNet, which is used only in the compression-efficiency ablation of Subsection 5.5) also has its original, uncompressed variant reported as a reference baseline in its corresponding table; this yields 84 baselines in total. We adjusted the hyperparameters (the number of basis filters and the number of coefficients to be preserved) to align with other methods

for a comparable compression rate, facilitating an analysis of the trade-off between compression rate and accuracy, and vice versa.

4.2. Classification on CIFAR-10

4.2.1. VGG-16

Table 2 reports the results of VGG-16 on CIFAR-10. SaP consistently outperforms weight-sharing approaches, including PENNI (Li et al., 2020a), Wong et al. (2024), SCWC (Li et al., 2022), Basis (Li et al., 2019), LegoNet (Yang et al., 2019), and GAP (Chen et al., 2024a). In particular, compared with PENNI (Li et al., 2020a), a kernel-sharing method, SaP achieves +1.31% accuracy with 39% fewer parameters and 21% lower computational cost. Against SCWC (Li et al., 2022), a clustering-based structured weight-sharing technique, SaP improves accuracy by 0.4% while using 43% fewer parameters. Relative to the basis scaling and double pruning method of Wong et al. (2024), SaP attains higher accuracy (+0.5%) while further reducing model size and computation by 53% and 24%, respectively. When compared to state-of-the-art pruning methods such as FPWT (Liu et al., 2024b), BCNet (Su et al., 2023), HSC (Lin et al., 2025), or the approaches of Ji et al. (2025) and Tang et al. (2025), SaP demonstrates superior compression in terms of both MACs and parameters, while still achieving higher accuracy. For instance, relative to HSC (Lin et al., 2025), a high-order spectral clustering-based filter pruning method, SaP delivers improvements in both accuracy (0.16% higher) and compression (11% fewer MACs and 6% fewer parameters). SaP also surpasses low-rank approximation techniques such as LRPET (Guo et al., 2025), RR-Tu2 (Dai et al., 2025), and ALDS (Liebenwein et al., 2021), delivering better accuracy with fewer parameters and computational overheads. Furthermore, against hybrid approaches such as the pruning-decomposition method of Dai et al. (2021), the pruning-distillation method BFP (Zhang & Liu, 2025), and HALOC (Xiao et al., 2023), a very competitive method combining NAS and tensor decomposition, SaP achieves comparable compression with better accuracy. For example, both SaP and HALOC (Xiao et al., 2023) reduce parameters by 98%, yet SaP attains 93.25% accuracy versus 93.16% for HALOC (Xiao et al., 2023). Finally, under an extreme compression setting of 99% reduction in both MACs and parameters, SaP remains robust, incurring only a modest accuracy drop of 3.6% compared to the original model.

4.2.2. ResNet-56/110

Table 3 presents the compression outcomes for ResNet-56 on CIFAR-10. We observe that hybrid approaches, which combine sharing and pruning, consistently outperform methods relying solely on one strategy across multiple compression settings. At moderate compression, pure pruning methods such as ACLI (Tofigh et al., 2026a) achieve moderate efficiency gains but limited accuracy improvement, while weight-sharing methods like SCWC (Li et al., 2022) and PENNI (Li et al., 2020a) achieve stronger reductions yet remain constrained in balancing compression with accuracy. Hybrid strategies such as G-SharP (Shin et al., 2024), which proposes to globally share filter bases across all layers, demonstrate the benefit of combining the two, reaching 94.16% accuracy with 50% parameter reduction. However, our SaP method advances this hybrid paradigm: with the same parameter reduction (50%), SaP delivers higher accuracy (94.30%) and requires fewer MACs (82.85M vs. 84.69M), showing superior accuracy–efficiency trade-offs. When targeting a higher compression ratio (51% MACs, 60% parameters), SaP attains 94.14% accuracy, again outperforming SCWC (Li et al., 2022), UDSP (Gao et al., 2024), Basis (Li et al., 2019), and CEPD (Sui et al., 2025), with both better accuracy and stronger parameter reduction. At extreme compression levels (66% MACs, 83% parameters), SaP still preserves 93.82% accuracy, surpassing G-SharP (Shin et al., 2024), GAP (Chen et al., 2024a), and HSC (Lin et al., 2025), showing that our method remains robust even under aggressive compression. Furthermore, SaP outperforms other compound decomposition–pruning approaches such as NORTON (Pham et al., 2025a), CEPD (Sui et al., 2025),

Table 2

Compression results of VGG-16 on CIFAR-10.

Method	Type	Top-1	MACs (RR)	Params (RR)
VGG-16 (Simonyan & Zisserman, 2015)		93.96	314.69M (00)	14.99M (00)
PENNI (Li et al., 2020a)	S + P	93.14	183.40M (42)	8.33M (44)
LRPET (Guo et al., 2025)	L	93.76	137.78M (56)	5.79M (61)
FPWT (Liu et al., 2024b)	P	93.94	140.28M (55)	3.96M (73)
Wong et al. (2024)	S + P	93.95	154.20M (51)	5.55M (63)
SCWC (Li et al., 2022)	S	94.03	182.83M (42)	9.07M (40)
BFP (Zhang & Liu, 2025)	P + K	94.13	128.42M (59)	4.07M (73)
BCNetV2 (Su et al., 2023)	N + P + K	94.36	138.00M (56)	3.30M (78)
Ji et al. (2025)	P	94.36	123.99M (61)	2.67M (82)
SaP	S + P	94.45	117.05M (63)	2.58M (83)
LegoNet (Yang et al., 2019)	S	91.97	74.70M (76)	1.90M (87)
SCWC (Li et al., 2022)	S	93.29	93.46M (70)	4.59M (69)
GAP (Chen et al., 2024a)	S + P	93.49	89.94M (71)	3.86M (74)
Fan et al. (2026)	P	93.49	N/A	1.58M (89)
Tang et al. (2025)	P	93.67	65.94M (79)	1.77M (88)
HSC (Lin et al., 2025)	P	93.75	96.31M (69)	2.09M (86)
Basis (Li et al., 2019)	S	93.77	73.95M (77)	2.99M (80)
SaP	S + P	93.91	62.76M (80)	1.20M (92)
LegoNet (Yang et al., 2019)	S	91.35	74.70M (76)	0.90M (94)
RR-Tu2 (Dai et al., 2025)	L	92.53	49.90M (84)	1.46M (90)
ALDS (Liebenwein et al., 2021)	L	92.67	43.33M (86)	0.63M (96)
BFP (Zhang & Liu, 2025)	P + K	92.89	51.31M (84)	2.67M (82)
Dai et al. (2021)	L + P	93.03	37.76M (87)	0.43M (97)
HALOC (Xiao et al., 2023)	L + N	93.16	43.92M (86)	0.30M (98)
SaP	S + P	93.25	33.99M (89)	0.30M (98)
SaP	S + P	90.36	4.52M (99)	0.14M (99)

Table 3

Compression results of ResNet-56 on CIFAR-10.

Method	Type	Top-1	MACs (RR)	Params (RR)
ResNet-56 (He et al., 2016)		93.26	126.87M (00)	855.77K (00)
SCWC (Li et al., 2022)	S	93.36	85.51M (33)	571.65K (33)
BFP (Zhang & Liu, 2025)	P + K	93.81	88.40M (30)	600.75K (30)
ACLI (Tofigh et al., 2026a)	P	93.84	86.85M (31)	595.36K (30)
PENNI (Li et al., 2020a)	S + P	94.00	92.80M (26)	471.00K (45)
G-SharP (Shin et al., 2024)	S + P	94.16	84.69M (33)	429.23K (50)
SaP	S + P	94.30	82.85M (35)	426.76K (50)
SCWC (Li et al., 2022)	S	93.16	73.84M (42)	487.79K (43)
ACLI (Tofigh et al., 2026a)	P	93.54	62.72M (51)	428.14K (50)
UDSP (Gao et al., 2024)	P	93.78	63.31M (50)	684.62K (20)
Basis (Li et al., 2019)	S	93.92	63.44M (50)	436.44K (49)
NORTON (Pham et al., 2025a)	L + P	94.00	73.22M (42)	447.83K (48)
SaP	S + P	94.14	62.32M (51)	343.24K (60)
Lee and Lee (2026)	P + L	93.16	43.91M (65)	255.28K (70)
G-SharP (Shin et al., 2024)	S + P	93.42	50.85M (60)	149.25K (83)
DPFP (Gao et al., 2026)	P	93.64	51.64M (59)	342.31K (60)
HSC (Lin et al., 2025)	P	93.79	50.32M (60)	341.45K (60)
CEPD (Sui et al., 2025)	L + P	93.70	59.50M (53)	311.50K (64)
GAP (Chen et al., 2024a)	S + P	93.72	45.34M (64)	330.58K (61)
SaP	S + P	93.82	43.37M (66)	148.72K (83)

and Lee and Lee (2026), consistently achieving higher accuracy while attaining greater compression.

For ResNet-110, similar trends are observed in Table 4. At moderate compression ratios, pruning-based methods such as ACLI (Tofigh et al., 2026a), HSC (Lin et al., 2025), FPWT (Liu et al., 2024b), and Ji et al. (2025) achieve accuracies around 94.0–94.4% while reducing parameters by approximately 60–65%. Notably, our SaP method delivers the highest accuracy (94.43%) while attaining the strongest compression (66% MACs, 62% parameters), surpassing the regularization-based network slimming method of Ji et al. (2025) in terms of accuracy–efficiency balance. Compared with the NAS–pruning-distillation hybrid approach of Lu et al. (2023), SaP offers slightly better accuracy (94.43% vs. 94.32%) while achieving more aggressive compression (86.67M vs. 108.80M MACs). When targeting more aggressive compression levels (75% MACs, 74% parameters), pruning-only strategies such as

Table 4
Compression results of ResNet-110 on CIFAR-10.

Method	Type	Top-1	MACs (RR)	Params (RR)
<i>ResNet-110</i> (He et al., 2016)		93.50	256.04M (00)	1730.71K (00)
LIAP (Tofigh et al., 2026b)	P	93.91	101.09M (60)	697.48K (60)
HSC (Lin et al., 2025)	P	94.01	88.26M (65)	692.28K (60)
FPWT (Liu et al., 2024b)	P	94.03	101.12M (60)	680.17K (61)
Lu et al. (2023)	N + P + K	94.32	108.80M (58)	N/A
Ji et al. (2025)	P	94.39	89.36M (65)	662.86K (62)
SaP	S + P	94.43	86.67M (66)	658.24K (62)
ACLI (Tofigh et al., 2026a)	P	93.28	77.28M (69)	533.06K (69)
SFI-FP (Yang et al., 2024)	P	93.41	67.78M (73)	488.06K (72)
HSC (Lin et al., 2025)	P	93.56	71.31M (72)	519.21K (70)
Ji et al. (2025)	P	93.69	71.69M (72)	508.83K (71)
DPFP (Gao et al., 2026)	P	93.71	68.36M (73)	539.98K (69)
SaP	S + P	93.71	64.79M (75)	449.17K (74)

SFI-FP (Yang et al., 2024), HSC (Lin et al., 2025), and Ji et al. (2025) all suffer a drop in accuracy to around 93.4–93.7%. SaP maintains competitive accuracy (93.71%) with consistently stronger compression (64.79M MACs, 449.17K parameters), demonstrating its robustness under extreme pruning.

Overall, these results confirm that our hybrid sharing–pruning strategy scales effectively to deeper networks, consistently outperforming state-of-the-art network compression baselines across a wide range of compression regimes.

4.2.3. DenseNet-40

Table 5 reports the compression results for DenseNet-40 on CIFAR-10. At moderate compression levels (MACs reduction around 40–46% and parameter reduction around 41–53%), pruning-only methods such as ARPruning (Yuan et al., 2024), FPWT (Liu et al., 2024b), and SPSRC (Sun & Shi, 2024) achieve accuracies in the range of 94.42–94.71%. Hybrid decomposition–pruning approaches like NORTON (Pham et al., 2025a) further improve accuracy, reaching 94.67% while maintaining similar compression rates. Compared with Wong et al. (2024), which achieves 94.72% accuracy with more limited compression (36% MACs and 38% parameter reduction), SaP delivers both higher accuracy and stronger compression. Our SaP method, which jointly applies sharing and pruning, achieves the same top-1 accuracy as the uncompressed DenseNet-40 (94.81%) while obtaining stronger compression (46% MACs reduction and 53% parameter reduction), demonstrating the advantage of combining basis sharing with pruning. At more aggressive compression levels, pruning-based methods such as EvoFC (Chen et al., 2024b), HRank (Lin et al., 2020a), and FPWT (Liu et al., 2024b) experience notable accuracy degradation, dropping to 93.2–93.8%, even when achieving significant MACs and parameter reductions. The attention map ranking approach, ARPruning (Yuan et al., 2024), mitigates some of this loss, achieving 94.20% accuracy at extreme compression (61% MACs reduction, 74% parameter reduction). Meanwhile, SaP maintains higher accuracy in this regime (94.23%) while achieving even stronger compression (68% MACs reduction, 74% parameter reduction), indicating that our hybrid sharing–pruning strategy is highly effective and robust for DenseNet, a densely connected architecture where each layer receives inputs from all previous layers. Overall, SaP demonstrates superior or competitive performance across both moderate and extreme compression regimes, consistently providing the best trade-off between accuracy and computational efficiency.

4.2.4. GoogLeNet

We validate the versatility of our method on a multi-branch architecture, GoogLeNet, with results reported in Table 6. At moderate compression levels (MACs reduction of 55–61% and parameter reduction of 54–58%), pruning-based methods such as HRank (Lin et al., 2020a), FPWT (Liu et al., 2024b), FSIM-E (Liu et al., 2023), and CORING (Pham et al., 2024a) achieve accuracies between 94.5–95.3%. Our

Table 5
Compression results of DenseNet-40 on CIFAR-10.

Method	Type	Top-1	MACs (RR)	Params (RR)
<i>DenseNet-40</i> (Huang et al., 2017)		94.81	290.14M (00)	1059.30K (00)
ARPruning (Yuan et al., 2024)	P	94.42	168.42M (41)	557.19K (46)
FPWT (Liu et al., 2024b)	P	94.44	165.36M (44)	586.39K (44)
RSDTR (Gabor & Zdunek, 2026)	L	94.51	161.19M (44)	504.23K (52)
NORTON (Pham et al., 2025a)	L + P	94.67	168.23M (42)	582.62K (45)
SPSRC (Sun & Shi, 2024)	P	94.69	168.82M (42)	593.21K (44)
CORING (Pham et al., 2023)	P	94.71	173.39M (40)	624.98K (41)
Wong et al. (2024)	S + P	94.72	185.68M (36)	656.76K (38)
SaP	S + P	94.81	156.63M (46)	502.54K (53)
EvoFC (Chen et al., 2024b)	N + P	93.23	134.99M (54)	503.16K (53)
HRank (Lin et al., 2020a)	P	93.68	110.15M (61)	487.28K (54)
FPWT (Liu et al., 2024b)	P	93.81	127.00M (57)	407.83K (44)
NORTON (Pham et al., 2025a)	L + P	94.17	103.68M (64)	328.38K (69)
ARPruning (Yuan et al., 2024)	P	94.20	108.81M (61)	275.42K (74)
RSDTR (Gabor & Zdunek, 2026)	L	94.21	120.89M (58)	407.83K (62)
SaP	S + P	94.23	93.77M (68)	273.21K (74)

Table 6
Compression results of GoogLeNet on CIFAR-10.

Method	Type	Top-1	MACs (RR)	Params (RR)
<i>GoogLeNet</i> (Szegedy et al., 2015)		95.05	1.52B (00)	6.15M (00)
EvoFC (Chen et al., 2024b)	N + P	94.28	0.71B (53)	3.44M (44)
HRank (Lin et al., 2020a)	P	94.53	0.69B (55)	2.74M (55)
FPWT (Liu et al., 2024b)	P	95.17	0.61B (60)	2.73M (55)
BFP (Zhang & Liu, 2025)	P + K	95.20	0.62B (60)	2.61M (58)
FSIM-E (Liu et al., 2023)	P	95.32	0.86B (43)	3.26M (47)
CORING (Pham et al., 2024a)	P	95.32	0.65B (57)	2.85M (54)
SAESA (Kumar et al., 2025)	N + P	95.37	0.60B (60)	N/A
SLIMING (Pham et al., 2025b)	P	95.48	0.62B (60)	2.68M (57)
SaP	S + P	95.48	0.59B (61)	2.57M (58)
DPFP (Gao et al., 2026)	P	94.74	0.42B (72)	2.18M (65)
BFP (Zhang & Liu, 2025)	P + K	94.89	0.41B (73)	1.90M (70)
FSIM-E (Liu et al., 2023)	P	94.92	0.43B (72)	2.19M (64)
DCFF (Lin et al., 2023)	P	94.92	0.46B (70)	2.08M (66)
CORING (Pham et al., 2024a)	P	95.03	0.39B (74)	2.10M (66)
FNP (Li et al., 2026)	N + P	95.03	0.51B (67)	2.11M (66)
ICP (Chang et al., 2023)	P	95.09	0.44B (71)	2.03M (67)
SaP	S + P	95.32	0.35B (77)	1.66M (73)

SaP method attains the highest top-1 accuracy (95.48%), matching the best-performing baseline, SLIMING (Pham et al., 2025b), a spectral-analysis-based pruning approach, while achieving stronger compression (61% MACs and 58% parameter reduction). This demonstrates that combining basis sharing with pruning is not only competitive with state-of-the-art methods but also surpasses them in compression efficiency without sacrificing accuracy. At more aggressive compression levels (MACs reduction of 70–77% and parameter reduction of 64–73%), pruning methods such as FSIM-E (Liu et al., 2023), DCFF (Lin et al., 2023), and ICP (Chang et al., 2023) sustain accuracies near 94.9–95.1%. In contrast, our SaP consistently outperforms these baselines, reaching 95.32% accuracy while achieving the strongest compression (77% MACs and 73% parameter reduction). Notably, in both levels of compression, SaP outperforms BFP (Zhang & Liu, 2025), an advanced hybrid method combining pruning and knowledge distillation. These results highlight the robustness of SaP under aggressive compression and further confirm its adaptability to complex multi-branch topologies like GoogLeNet.

4.3. Classification on CIFAR-100

4.3.1. VGG-16

Table 7 displays the compression outcomes for VGG-16 on CIFAR-100. At moderate compression levels, pruning-based methods such as SPSRC (Sun & Shi, 2024), FPC (Chen et al., 2022), and HTP-URC (Qian et al., 2024) achieve accuracies ranging from 73.16% to 74.03% while reducing MACs by 33–44% and parameters by 65–88%. Our SaP method not only surpasses these approaches by attaining the best accuracy

Table 7

Compression results of VGG-16 on CIFAR-100.

Method	Type	Top-1	MACs (RR)	Params (RR)
VGG-16 (Simonyan & Zisserman, 2015)		73.45	314.62M (00)	15.04M (00)
SPSRC (Sun & Shi, 2024)	P	73.16	211.64M (33)	5.29M (65)
FPC (Chen et al., 2022)	P	73.44	175.63M (44)	1.83M (88)
HTP-URC (Qian et al., 2024)	P	74.03	179.63M (43)	2.82M (81)
SaP	S + P	74.25	164.64M (48)	1.73M (89)
RR-Tu2 (Dai et al., 2025)	L	67.83	50.10M (84)	2.36M (84)
CoDeC (Pham et al., 2026)	L + S	68.46	49.87M (84)	2.18M (86)
RGP (Chen et al., 2024c)	P	69.33	49.55M (84)	2.39M (84)
Tang et al. (2025)	P	70.54	47.82M (85)	1.63M (89)
SLIMING (Pham et al., 2025b)	P	72.18	44.88M (86)	0.85M (94)
SaP	S + P	72.21	39.69M (87)	0.66M (96)

(74.25%) but also achieves stronger compression, with 48% MACs reduction and 89% parameter reduction. This highlights the effectiveness of combining basis sharing with pruning in striking a better trade-off between efficiency and accuracy. At more aggressive compression levels (84–87% MACs reduction and 84–96% parameter reduction), several methods such as RR-Tu2 (Dai et al., 2025), CoDeC (Pham et al., 2026), RGP (Chen et al., 2024c), and Tang et al. (2025) have leveraged tensor decomposition and pruning for aggressive compression. Notably, CoDeC (Pham et al., 2026), recently published as a strong baseline, combines coupled tensor decomposition with weight sharing by enforcing a joint factorization across similar filters. Despite the strength of this approach, our SaP method achieves superior results: under a comparable compression ratio, SaP attains a higher top-1 accuracy (72.21% vs. 68.46%) while further reducing both MACs (39.69M vs. 49.87M) and parameters (0.66M vs. 2.18M). These results underscore the effectiveness of integrating basis sharing with coefficient pruning, establishing SaP as a more competitive solution than decomposition-based baselines.

4.3.2. ResNet-20/32/56/110

Table 8 reports the compression results on CIFAR-100 with ResNet-20/32/56/110.

ResNet-20. SaP outperforms both tensor decomposition and structured pruning approaches (Dai et al., 2025; Jayasimhan & Pabitha, 2024; Pham et al., 2025b) in terms of accuracy preservation and parameter efficiency. Specifically, a closer comparison with SLIMING (Pham et al., 2025b), a recent singular values-driven automated pruning method, is particularly insightful. Both methods reach the same MACs reduction ratio (60%), yet SaP attains a slightly higher accuracy (67.92% vs. 67.83%) while yielding a more compact model in terms of parameters (96.74K vs. 102.33K, corresponding to an additional 2% savings). This improvement underscores the complementary effect of introducing basis sharing on top of pruning: whereas SLIMING leverages singular values to guide pruning decisions, SaP further reduces redundancy through shared basis representations, leading to better efficiency without sacrificing accuracy.

ResNet-32. Our method continues to deliver improvements over prior pruning approaches. Compared to DAIS (Guan et al., 2023), a channel pruning method based on differentiable annealing indicator search, SaP achieves a higher compression ratio (46% vs. 43%) while maintaining slightly better accuracy (72.40% vs. 72.20%). A direct comparison with SLIMING (Pham et al., 2025b) further emphasizes the advantage of our approach. Although both methods reach similar parameter reduction levels (45%), SaP yields a more compact model in terms of MACs (37.98M vs. 38.96M) while also achieving marginally higher accuracy (72.40% vs. 72.37%). This consistent gain demonstrates that combining pruning with basis sharing not only preserves accuracy but also enhances efficiency beyond what singular values-driven pruning alone can achieve.

ResNet-56. SaP demonstrates clear advantages over existing pruning methods. Compared to the network compression approach using graph theory proposed by Li et al. (2024), which achieves a 46% MACs reduc-

Table 8

Compression results of ResNet-20/32/56/110 on CIFAR-100.

Method	Auto	Top-1	MACs (RR)	Params (RR)
ResNet-20 (He et al., 2016)		68.92	41.25M (00)	278.32K (00)
RR-Tu2 (Dai et al., 2025)	L	66.30	17.40M (58)	138.60K (50)
ResPrune (Jayasimhan & Pabitha, 2024)	P	67.62	28.74M (58)	N/A
SLIMING (Pham et al., 2025b)	P	67.83	16.42M (60)	102.33K (63)
SaP	S + P	67.92	16.42M (60)	96.74K (65)
ResNet-32 (He et al., 2016)		72.16	69.78M (00)	472.76K (00)
DAIS (Guan et al., 2023)	P	72.20	39.40M (43)	N/A
SLIMING (Pham et al., 2025b)	P	72.37	38.96M (44)	261.66K (45)
SaP	S + P	72.40	37.98M (46)	258.94K (45)
ResNet-56 (He et al., 2016)		72.89	126.86M (00)	861.62K (00)
Li et al. (2024)	P	70.12	68.89M (46)	726.00K (16)
FGC (Shi et al., 2024)	P	72.00	74.76M (41)	N/A
HTP-URC (Qian et al., 2024)	P	71.45	95.15M (25)	636.74K (26)
ASTER (Zhang & Freris, 2024)	P	72.23	92.00M (27)	N/A
SaP	S + P	72.65	66.79M (47)	475.58K (45)
ResNet-110 (He et al., 2016)		74.42	255.29M (00)	1736.56K (00)
SLIMING (Pham et al., 2025b)	P	73.07	108.90M (57)	775.15K (55)
Lu et al. (2023)	N + P + K	73.12	108.00M (58)	N/A
SaP	S + P	73.48	107.62M (58)	727.92K (58)

tion but at the cost of a significant accuracy drop (70.12%), SaP attains a comparable or higher reduction (47%) while preserving much higher accuracy (72.65%). Relative to other recent pruning strategies such as FGC (Shi et al., 2024), HTP-URC (Qian et al., 2024), and ASTER (Zhang & Freris, 2024), our method consistently delivers a better balance between compression and performance. Specifically, SaP achieves the lowest MACs (66.79M) and the fewest parameters (475.58K, 45% reduction) while also reaching the highest accuracy among compressed models (72.65%). These results highlight the effectiveness of combining pruning with basis sharing, enabling SaP to simultaneously reduce redundancy and maintain accuracy even on deeper architectures.

ResNet-110. SaP maintains its advantage over recent approaches (Lu et al., 2023; Pham et al., 2025b). Compared to SLIMING (Pham et al., 2025b), which achieves a 57% MACs reduction and 55% parameter reduction, SaP delivers a higher top-1 accuracy (73.48% vs. 73.07%) while maintaining similar compression (58% MACs, 58% parameters). When compared with the NAS-pruning-distillation hybrid method of Lu et al. (2023), SaP achieves slightly better accuracy (73.48% vs. 73.12%) and a comparable reduction in computational cost (107.62M vs. 108.00M MACs). These results indicate that the joint sharing–pruning strategy of SaP effectively preserves accuracy while reducing both computational and memory overhead, demonstrating its scalability and robustness on deeper networks.

Overall, SaP consistently provides superior accuracy–efficiency trade-offs across both VGG and ResNet architectures, demonstrating the generality and effectiveness of combining basis sharing with pruning.

4.4. Classification on Imagenet

4.4.1. ResNet-18

Table 9 presents the compression results of ResNet-18 on ImageNet. We observe that hybrid compression strategies (Ganjanesh et al., 2024; Hou et al., 2024; Li et al., 2025; Phan et al., 2024; Sui et al., 2025) consistently outperform single-technique approaches (Guo et al., 2025; Li et al., 2022; Pham et al., 2025b). Methods that rely solely on pruning (Pham et al., 2025b), low-rank decomposition (Guo et al., 2025), or parameter sharing (Li et al., 2022) reduce computational cost but incur more severe accuracy degradation. In contrast, hybrid methods that combine complementary mechanisms (e.g., P + K, P + N, L + P, L + S, S + P) achieve a better accuracy–compression trade-off by exploiting multiple forms of redundancy. Within this category, SaP consistently delivers improvements over prior art. Relative to PEEL (Hou et al., 2024), which combines pruning and knowledge distillation, SaP

Table 9
Compression results of ResNet-18 on ImageNet.

Method	Type	Top-1	Top-5	MACs (RR)	Params (RR)
<i>ResNet-18</i> (He et al., 2016)		69.75	89.07	1.82B (00)	11.69M (00)
SCWC (Li et al., 2022)	S	67.27	87.54	0.92B (49)	6.00M (49)
SFP (Li et al., 2025)	P+K	67.29	87.57	1.10B (40)	7.29M (38)
RSDTR (Gabor & Zdunek, 2026)	L	69.87	89.28	1.38B (24)	6.32M (46)
PEEL (Hou et al., 2024)	P+K	69.90	89.33	1.04B (43)	6.84M (41)
SaP	S+P	70.08	89.87	0.92B (49)	5.89M (50)
LRPET (Guo et al., 2025)	L	67.87	88.04	0.86B (53)	5.81M (50)
FGFP (Tu et al., 2025)	P+S	68.61	89.08	N/A	2.32M (60)
RLAL (Ganjanesh et al., 2024)	P+N	69.80	89.10	0.98B (50)	N/A
CEPD (Sui et al., 2025)	L+P	69.86	89.32	0.89B (51)	4.75M (59)
SaP	S+P	69.91	89.37	0.84B (54)	4.67M (60)
RSDTR (Gabor & Zdunek, 2026)	L	68.76	88.59	0.89B (51)	3.91M (67)
SLIMING (Pham et al., 2025b)	P	68.82	88.75	0.75B (59)	3.15M (73)
SSC (Phan et al., 2024)	L+S	68.89	88.77	0.73B (60)	3.11M (73)
SaP	S+P	69.15	89.01	0.69B (62)	3.05M (74)

achieves higher top-1 accuracy (70.08% vs. 69.90%) while yielding larger reductions in MACs (49% vs. 43%). Against the structured pruning and low-rank tensor decomposition approach CEPD (Sui et al., 2025), SaP maintains similar accuracy (69.91% vs. 69.86%) and secures slightly greater savings in MACs (54% vs. 51%) and parameters (54% vs. 51%), underscoring the efficiency of basis sharing. Finally, compared to SSC (Phan et al., 2024), which uses block-term decomposition with shared core tensors, SaP achieves a clear accuracy gain (69.15% vs. 68.89%) while also providing improved compression. Overall, these results confirm that hybrid designs are preferable to single techniques and that the integration of basis sharing with coefficient pruning in SaP provides a superior balance between compactness and performance.

4.4.2. ResNet-34

Table 10 reports the compression outcomes on ResNet-34. In the moderate compression regime, SaP achieves the best balance between accuracy and efficiency, reaching 73.36% top-1 with 54% reductions in both MACs and parameters. This clearly outperforms decomposition-driven methods such as EDP (Ruan et al., 2021) and LRPET (Guo et al., 2025). Compared to the cross-layer shared filter bases method of Kang and Kim (2021), SaP attains similar accuracy while delivering nearly five times greater MACs reduction (54% vs. 11%). Under more aggressive settings, the accuracy gap becomes sharper. The coupled tensor train decomposition approach NC_CTD (Sun et al., 2021) and the joint matrix factorization approach Bi-JSVD (Chen et al., 2023) suffer drastic accuracy drops, with top-1 scores below 64%. This degradation stems from their reliance on shared factor matrices across layers, which reduces redundancy but also weakens the model's representational capacity. By contrast, SaP combines layer-wise basis sharing with pruning, preserving critical information while discarding redundant coefficients. As a result, SaP achieves approximately 10% and 8% higher accuracy than NC_CTD (Sun et al., 2021) and Bi-JSVD (Chen et al., 2023), respectively, at similar compression levels. These findings suggest that hybrid basis sharing with pruning awareness provides a more flexible and reliable pathway to extreme compression than coupled decompositions.

4.4.3. ResNet-50

Table 11 reports the compression results of ResNet-50. At a moderate compression level, SaP delivers the best trade-off between accuracy and efficiency, achieving 76.53% top-1 accuracy with 47% reductions in both MACs and parameters. Compared to single-technique baselines, SaP shows clear advantages. The pruning-only method ACLI (Tofigh et al., 2026a) reaches only 75.37% top-1 under a similar compression ratio, while the tensor decomposition method RR-Tu2 (Dai et al., 2025) preserves accuracy (76.10%) but falls short in efficiency, reducing parameters by just 33% compared to 47% for SaP. The cross-layer pa-

Table 10
Compression results of ResNet-34 on ImageNet.

Method	Type	Top-1	Top-5	MACs (RR)	Params (RR)
<i>ResNet-34</i> (He et al., 2016)		73.30	91.42	3.67B (00)	21.80M (00)
EDP (Ruan et al., 2021)	L+P	72.33	90.88	2.02B (44)	11.87M (46)
ATO (Wu et al., 2024)	P	72.92	91.15	2.05B (44)	N/A
LRPET (Guo et al., 2025)	L	72.95	91.16	1.71B (53)	10.51M (52)
RSDTR (Gabor & Zdunek, 2026)	L	72.99	91.22	3.10B (15)	11.90M (45)
BCNetV2 (Su et al., 2023)	N+P+K	73.30	91.40	1.84B (50)	16.90M (22)
RLAL (Ganjanesh et al., 2024)	P+N	73.31	91.25	1.85B (50)	N/A
Kang and Kim (2021)	S	73.33	91.46	3.26B (11)	11.79M (46)
SaP	S+P	73.36	91.51	1.70B (54)	10.12M (54)
NC_CTD (Sun et al., 2021)	L+S	61.12	84.26	1.15B (69)	3.79M (83)
EvoFC (Chen et al., 2024b)	N+P	63.10	85.79	1.05B (71)	9.96M (54)
Bi-JSVD (Chen et al., 2023)	L+S	64.12	86.16	1.07B (71)	3.79M (83)
SaP	S+P	72.05	91.12	0.97B (74)	3.77M (83)
NC_CTD (Sun et al., 2021)	L+S	58.37	82.29	0.99B (73)	1.99M (91)
EvoFC (Chen et al., 2024b)	N+P	58.78	82.36	0.75B (80)	4.93M (77)
Bi-JSVD (Chen et al., 2023)	L+S	60.83	83.58	0.95B (74)	1.99M (91)
SaP	S+P	68.84	89.23	0.93B (75)	1.92M (91)

rameter sharing method of Kang and Kim (2021) does not scale effectively on deeper architectures, yielding a slight accuracy drop (−0.48%) and negligible MACs savings despite a 37% parameter reduction. Relative to hybrid strategies, SaP also establishes a consistent advantage. G-SharP (Shin et al., 2024) and PENNI (Li et al., 2020a), which combine sharing with pruning, deliver lower accuracy at higher computational cost. More sophisticated hybrids, such as Lu et al. (2023) and Lee and Song (2024), retain near-baseline accuracy but achieve smaller reductions in MACs (45% and 39%, respectively).

When we increase the compression to 64% of both MACs and parameters, SaP continues to show strong robustness, representing a competitive balance compared to other methods that operate in a similar regime. Among sole-technique baselines, pruning-only methods such as DCFE (Lin et al., 2023) still maintain reasonable accuracy, but SaP surpasses them (75.84% vs. 75.60%) while achieving identical computational savings. Pure parameter sharing approaches like LegoNet (Yang et al., 2019) and FSNet (Yang et al., 2020), on the other hand, collapse significantly, dropping more than 2% in top-1 accuracy. This confirms that weight sharing using Lego filters (Yang et al., 2019) or filter summary (Yang et al., 2020) techniques struggles to retain representation capacity under aggressive compression. Hybrid methods exhibit more resilience, but they generally sacrifice either accuracy or efficiency. For example, BFP (Zhang & Liu, 2025) and G-SharP (Shin et al., 2024) reach accuracies above 75% but at the expense of larger parameter counts or weaker MACs reduction. Notably, GAP (Chen et al., 2024a), a pruning via kernel fusion and sharing, matches SaP in MACs reduction and accuracy but still trails by 7% in parameter efficiency.

At an extreme compression regime of 77% MACs and 71% parameter reduction, SaP continues to demonstrate robustness, retaining 73.69% top-1 accuracy while delivering the strongest efficiency gains. Sole-technique baselines generally struggle in this setting: FSNet (Yang et al., 2020) collapses sharply to 70.36%, confirming again the fragility of pure sharing approaches. Among pruning-based methods, CHIP (Sui et al., 2021), SNACS (Ravi Ganesh et al., 2024), and RGP (Chen et al., 2024c) achieve accuracies between 72.3% and 72.7%, but none balance accuracy with compactness as effectively as SaP. Even the stronger pruning variant SFI-FP (Yang et al., 2024) falls behind, trailing by 0.21% top-1 accuracy while requiring more parameters and computation. LRPET (Guo et al., 2025), a compact model training method by low-rank projection with energy transfer, reduces parameters to a similar level but suffers nearly a 3% drop in accuracy compared to SaP.

Overall, SaP scales gracefully across moderate, high, and extreme compression, consistently outperforming both single-technique and

Table 11
Compression results of ResNet-50 on ImageNet.

Method	Type	Top-1	Top-5	MACs (RR)	Params (RR)
<i>ResNet-50</i> (He et al., 2016)		76.13	92.87	4.12B (00)	25.56M (00)
RGP (Chen et al., 2024c)	P	75.30	92.55	2.30 B (44)	14.34M (44)
ACLI (Tofigh et al., 2026a)	P	75.37	92.56	2.26 B (45)	14.61M (43)
SFP (Li et al., 2025)	P + K	75.39	92.56	2.20B (47)	14.88M (42)
G-SharP (Shin et al., 2024)	S + P	75.54	92.58	2.95B (28)	16.18M (37)
Kang and Kim (2021)	S	75.65	92.59	4.11B (00)	16.02M (37)
Lu et al. (2023)	N + P + K	76.04	92.83	2.25B (45)	N/A
RR-Tu2 (Dai et al., 2025)	L	76.10	92.97	2.64B (36)	17.00M (33)
PENNI (Li et al., 2020a)	S + P	76.20	92.85	3.23B (22)	N/A
Lee and Song (2024)	P + N + K	76.23	92.87	2.48B (39)	21.56M (15)
SaP	S + P	76.53	93.12	2.18B (47)	13.58M (47)
LegoNet (Yang et al., 2019)	S	73.06	91.30	2.01B (51)	9.30M (64)
FSNet (Yang et al., 2020)	S	73.11	91.37	N/A	13.90M (46)
G-SharP (Shin et al., 2024)	S + P	75.00	92.42	2.38B (42)	12.42M (51)
FNP (Li et al., 2026)	N + P	75.10	92.42	1.95B (53)	15.09M (41)
SFP (Li et al., 2025)	P + K	75.24	92.43	1.95B (53)	12.69M (50)
EDP (Ruan et al., 2021)	L + P	75.34	92.43	1.92B (53)	14.28M (44)
FGFP (Tu et al., 2025)	P + S	75.42	92.45	N/A	9.53M (63)
BFP (Zhang & Liu, 2025)	P + K	75.47	92.47	1.68B (59)	13.48M (47)
CORING (Pham et al., 2024a)	L + P	75.55	92.61	1.50B (64)	11.04M (57)
DCFF (Lin et al., 2023)	P	75.60	92.55	1.52B (63)	11.05M (57)
GAP (Chen et al., 2024a)	S + P	75.72	92.72	1.50B (64)	10.90M (57)
CEPD (Sui et al., 2025)	L + P	75.82	92.84	1.53B (63)	9.38M (63)
SaP	S + P	75.84	92.85	1.50B (64)	9.26M (64)
FSNet (Yang et al., 2020)	S	70.36	89.79	N/A	8.49M (67)
LRPET (Guo et al., 2025)	L	70.97	90.16	0.98B (76)	7.74M (70)
Hrank (Lin et al., 2020a)	P	72.30	90.74	0.95 B (77)	8.02M (69)
SNACS (Ravi Ganesh et al., 2024)	P	72.60	91.01	1.98 B (52)	7.92M (69)
RGP (Chen et al., 2024c)	P	72.68	91.06	0.94 B (77)	8.13M (68)
CHIP (Sui et al., 2021)	P	73.30	91.48	0.95 B (77)	8.01M (69)
SFI-FP (Yang et al., 2024)	P	73.48	91.55	0.96 B (77)	8.03M (69)
BFP (Zhang & Liu, 2025)	P + K	73.52	91.71	1.04B (75)	7.49M (71)
SaP	S + P	73.69	91.86	0.93B (77)	7.49M (71)

hybrid baselines in accuracy and compactness, establishing itself as a robust and versatile solution for compressing deep architectures.

4.4.4. MobileNetV2

Table 12 summarizes compression results of MobileNetV2. At moderate compression (~30% MACs reduction), hybrid strategies such as RLAL (Ganjdanesh et al., 2024) and Lu et al. (2023) achieve accuracies close to the original model, demonstrating the benefit of combining pruning with auxiliary mechanisms like knowledge distillation or reinforcement learning. Our method, SaP, further improves this trade-off by reaching the highest top-1 accuracy (72.03%) while matching the lowest computational cost (30% MACs reduction) and delivering the largest parameter savings (22%). As compression becomes more aggressive (~35–37% MACs reduction), purely pruning-based methods such as DCP (Liu et al., 2022) and Guo et al. (2024) maintain competitive accuracy but with either fewer parameter savings or higher computational demands. In contrast, SaP achieves 71.92% top-1 while reducing MACs by 37% and parameters by 36%, outperforming competing approaches (Chen et al., 2024b; Gao et al., 2024; Guo et al., 2024; Liu et al., 2022; Zhang & Freris, 2024) in this regime. At extreme compression, compared to GCPNet (Zhang et al., 2025), an advanced hybrid pruning method with a bilateral coupled sampling strategy, SaP achieves a similar MACs reduction but yields 1.05% higher accuracy with over five times greater parameter reduction (50% vs. 9%). These results highlight that SaP is not limited to large, highly redundant architectures, but also transfers effectively to compact backbones like MobileNetV2, making it a versatile solution for optimizing mobile architectures.

4.4.5. EfficientNet-B0

Table 13 reports compression results on ImageNet using EfficientNet-B0. Unlike ResNet-50 or MobileNetV2, EfficientNet-B0 is already a compact architecture optimized by neural architecture search (Tan & Le,

Table 12
Compression results of MobileNetV2 on Imagenet.

Method	Type	Top-1	MACs (RR)	Params (RR)
<i>MobileNetV2</i> (Sandler et al., 2018)		71.89	320.24M (00)	3.51M (00)
Niu et al. (2023)	P	69.40	229.50M (28)	2.85M (19)
Lu et al. (2023)	N + P + K	71.61	226.09M (29)	N/A
HTP-URC (Qian et al., 2024)	P	71.68	224.16M (30)	N/A
RLAL (Ganjdanesh et al., 2024)	P + N	71.82	226.09M (29)	N/A
ATO (Wu et al., 2024)	P + N	72.02	223.85M (30)	N/A
SaP	S + P	72.03	223.65M (30)	2.73M (22)
EvoFC (Chen et al., 2024b)	N + P	61.64	211.04M (34)	3.14M (11)
DCP (Liu et al., 2022)	P	71.39	222.02M (31)	2.29M (35)
Guo et al. (2024)	P	71.49	207.58M (35)	2.26M (35)
UDSP (Gao et al., 2024)	P	71.72	203.03M (37)	2.97M (15)
ASTER (Zhang & Freris, 2024)	P	71.89	221.00M (31)	N/A
SaP	S + P	71.92	200.52M (37)	2.24M (36)
SFP (Li et al., 2025)	P + K	67.89	200.15M (38)	2.68M (24)
DCFF (Lin et al., 2023)	P	68.60	149.55M (53)	2.62M (25)
GCPNet (Zhang et al., 2025)	N + P + S	68.70	112.48M (65)	3.20M (09)
Guo et al. (2024)	P	69.58	173.99M (46)	1.90M (46)
SLIMING (Pham et al., 2025b)	P	69.62	140.09M (53)	1.83M (48)
SaP	S + P	69.75	112.21M (65)	1.76M (50)

Table 13
Compression results of EfficientNet-B0 on Imagenet.

Method	Type	Top-1	Top-5	MACs (RR)	Params (RR)
<i>EfficientNet-B0</i> (Tan & Le, 2019)	N	76.88	92.64	385.83M (00)	5.29M (00)
BCNetV2 (Su et al., 2023)	N + P + K	75.24	92.28	192.48M (50)	3.82M (28)
GCPNet (Zhang et al., 2025)	N + P + S	75.60	92.50	192.48M (50)	3.64M (31)
SaP	S + P	75.63	92.51	191.93M (50)	3.62M (32)

2019), which makes further compression particularly challenging. In this setting, we compare SaP against two strong baselines, BCNetV2 (Su et al., 2023) and GCPNet (Zhang et al., 2025), both of which integrate pruning with NAS-based strategies to preserve accuracy under tight resource constraints. BCNetV2 (Su et al., 2023) employs a bilaterally coupled supernet to ensure fair training of different channel widths during the search process, while GCPNet (Zhang et al., 2025) introduces gradient-aware sampling and homogenization techniques to improve the reliability of subnet performance estimation. Despite these sophisticated designs, SaP achieves the best overall trade-off, reaching 75.63% top-1 accuracy and 92.51% top-5 accuracy with 50% MACs and 32% parameter reduction. This slightly outperforms both BCNetV2 and GCPNet in accuracy while delivering the most compact model. These results highlight that even for architectures already optimized by NAS, SaP provides complementary benefits by combining parameter sharing with structured pruning, demonstrating robustness and versatility across both large-scale and compact models.

4.4.6. Vision transformer

While the primary focus of SaP in this work is on convolutional architectures, the proposed basis-sharing and coefficient-pruning framework is not restricted to convolutional filters and can be naturally extended to other weight matrices. To explore its applicability to transformer-based architectures, we evaluate SaP on a vision transformer model.

Specifically, we consider DeiT (Touvron et al., 2021), a representative vision transformer architecture derived from ViT (Dosovitskiy et al., 2021) and enhanced with knowledge distillation for improved training efficiency on ImageNet. In transformer blocks, the dominant computational cost arises from the linear projection layers in the multi-head self-attention modules and feed-forward networks. Since these components consist of structured weight matrices similar to convolutional weight tensors when unfolded into matrix form, SaP can be directly applied by performing basis sharing on these linear layers followed by coefficient pruning.

Table 14 presents the compression results obtained with SaP and compares them with three representative transformer compression

Table 14

Compression results of vision transformer (DeiT) on Imagenet.

Method	Type	Top-1	MACs (RR)	Params (RR)
DeiT (Touvron et al., 2021)	K	72.20	1264M (00)	5717K (00)
S ² ViTE-T (Chen et al., 2021)	P	70.12	965M (24)	4210K (26)
SVD (Hajimolahoseini et al., 2022)	L	70.06	911M (28)	4059K (29)
UPDP (Liu et al., 2024a)	P + N	70.30	900M (29)	3800K (34)
SaP	S + P	70.41	846M (33)	3774K (34)

Table 15

Results of Faster/Mask/Keypoint-RCNN on COCO-2017.

Model	Type	AP _{50:95}	AP ₅₀	MACs (RR)	Params (RR)
FasterRCNN (Ren et al., 2017)		36.91	58.54	134.85 B (00)	41.81M (00)
NORTON (Pham et al., 2025a)	L + P	32.47	52.05	93.39 B (31)	22.01M (47)
SaP	S + P	35.07	55.81	90.84 B (33)	21.25M (49)
MaskRCNN (He et al., 2020)		34.52	56.02	134.85 B (00)	44.46M (00)
CORING (Pham et al., 2024a)	L + P	32.77	53.53	92.23 B (32)	26.68M (40)
SaP	S + P	32.95	53.96	90.84 B (33)	24.81M (44)
KeypointRCNN (He et al., 2020)		65.06	86.11	137.42 B (00)	59.64M (00)
CoDeC (Pham et al., 2026)	L + S	63.62	85.13	90.56 B (34)	39.90M (33)
SaP	S + P	63.64	85.16	90.11 B (34)	39.75M (33)

approaches. S²ViTE-T (Chen et al., 2021) is a sparsity-aware training framework that jointly learns structured sparse subnetworks during training, including attention-head sparsity and adaptive token selection to reduce computation. The SVD-based approach (Hajimolahoseini et al., 2022) applies progressive low-rank decomposition to transformer weight matrices during fine-tuning, enabling gradual rank reduction with improved accuracy retention compared with single-shot decomposition. UPDP (Liu et al., 2024a) is a progressive depth pruning method that combines block-level pruning with supernet-based subnet searching using an evolutionary optimization strategy to identify efficient architectures under depth constraints.

As shown in Table 14, SaP achieves competitive performance among these approaches, obtaining a Top-1 accuracy of 70.41% while reducing MACs by 33% and parameters by 34%. Compared with sparsity-based approaches such as S²ViTE-T (Chen et al., 2021), which learn sparse subnetworks during training, SaP achieves stronger compression at similar accuracy levels while operating as a post-training compression method, suggesting that combining basis sharing with coefficient pruning provides an effective mechanism for redundancy removal beyond structured sparsification alone. Compared with progressive low-rank decomposition methods such as SVD (Hajimolahoseini et al., 2022), SaP achieves both higher accuracy and greater computational reduction, indicating that the hybrid sharing-pruning strategy provides a flexible alternative to purely factorization-based compression. Furthermore, SaP achieves comparable or better performance than UPDP (Liu et al., 2024a) while operating as a lightweight post-training compression method that does not require supernet construction or architecture-level subnet search.

These results suggest that SaP can be effectively extended to transformer-based architectures by operating on linear projection layers within attention and feed-forward modules. Nevertheless, it is important to note that transformer models exhibit different redundancy patterns compared with convolutional networks, particularly due to attention-head interactions and token-mixing operations. Therefore, while the present results demonstrate promising applicability, further investigation of layer-wise adaptation strategies and attention-specific compression mechanisms constitutes an interesting direction for future work.

4.5. Object detection, instance segmentation, and keypoint detection on COCO

4.5.1. Faster/mask/keypoint-RCNN

To evaluate the applicability of SaP to downstream tasks, we compress Faster-RCNN, Mask-RCNN, and Keypoint-RCNN on COCO

Table 16

Results of RT-DETR on COCO-2017.

Model	AP _{50:95}	AP ₅₀	MACs (RR)	Params (RR)
RT-DETR (Zhao et al., 2024)	53.1	71.3	138.04 B (00)	42.86M (00)
YOLOv7 (Wang et al., 2023)	51.2	69.7	104.70 B (24)	36.90M (14)
SaP	52.1	70.5	107.60 B (22)	33.30M (22)

Table 17Complexity reductions, approximation error, and accuracy without and with fine-tuning (FT) with respect to $\frac{B^{(0)}}{O^{(0)}}$.

$\frac{B^{(0)}}{O^{(0)}}$	MACs	Params	NMSE	Δ accuracy	
	RR	RR		No FT	FT
1/8	85.59	84.35	0.2859	-83.69	-0.12
2/8	71.48	70.58	0.1803	-29.52	0.48
3/8	57.34	56.81	0.1172	-6.52	1.24
4/8	43.24	43.03	0.0775	-3.14	1.30
5/8	29.13	29.26	0.0484	-0.79	1.48
6/8	14.99	15.49	0.0275	-0.44	1.69
7/8	0.88	1.72	0.0117	-0.13	1.68

(Table 15). For Faster-RCNN, SaP achieves 35.07 AP_{50:95} and 55.81 AP₅₀, reducing MACs by 33% and parameters by 49%. This corresponds to a +2.60 AP_{50:95} improvement over NORTON (Pham et al., 2025a) under a similar compute budget, highlighting SaP's superior robustness at high compression. For Mask-RCNN, compared to CORING (Pham et al., 2024a), SaP yields +0.18 AP_{50:95} with slightly stronger compression (33% vs. 32% MACs reduction and 44% vs. 40% parameter reduction), confirming its adaptability to the more challenging instance segmentation task. Finally, for Keypoint-RCNN, SaP performs on par with CoDeC (Pham et al., 2026), reaching 63.64 AP_{50:95} and 85.16 AP₅₀ with essentially the same compression levels (34% MACs and 33% parameter reduction). These tasks are substantially more complex than image classification, as they require not only recognition but also precise localization and structured predictions. Despite this increased difficulty, SaP consistently delivers competitive accuracy while providing stronger or comparable compression than recent methods, demonstrating its generalization across detection, segmentation, and keypoint estimation.

4.5.2. Hybrid CNN-transformer architecture: RT-DETR

In addition to pure convolutional and transformer architectures, we further evaluate the applicability of SaP on hybrid CNN-transformer models. Specifically, we consider RT-DETR (Zhao et al., 2024), a real-time end-to-end object detection framework that combines a convolutional backbone for hierarchical feature extraction with a transformer-based encoder-decoder detection head. Such hybrid architectures are widely adopted in modern detection pipelines and therefore provide a representative testbed for assessing the generality of SaP beyond single-family network designs.

In this experiment, SaP is applied to both the convolutional backbone as well as the transformer encoder and decoder RT-DETR, demonstrating that the proposed basis-sharing and coefficient-pruning strategy can operate consistently across heterogeneous architectural components within the same model. Table 16 summarizes the compression results on the COCO benchmark. Compared with the original RT-DETR (Zhao et al., 2024), SaP reduces parameters by 22% and computational cost by 22% while maintaining competitive detection accuracy, with only a -1.0 AP_{50:95} decrease.

We further compare the compressed RT-DETR model with YOLOv7 (Wang et al., 2023), a representative real-time convolutional detection architecture. Despite operating under a comparable computational reduction regime (22% vs. 24% MACs reduction), the SaP-compressed RT-DETR achieves higher detection accuracy (+0.9 AP_{50:95} and +0.8 AP₅₀) while also providing stronger parameter savings (22% vs. 14%). These results suggest that SaP remains effective when applied

to hybrid architectures that combine convolutional feature extractors with transformer-based reasoning modules.

Overall, this experiment complements the DeiT results presented in Subsubsection 4.4.6 and further demonstrates that SaP can be flexibly deployed across convolutional, transformer-based, and hybrid CNN–transformer architectures. Nevertheless, since hybrid detection frameworks involve additional components such as cross-scale feature fusion and query-based decoding, further investigation of component-wise compression strategies within end-to-end detection pipelines remains an interesting direction for future work.

5. Ablation studies and discussions

5.1. On the selection of the number of basis filters

As the number of basis filters B directly affects the compression ratio, approximation error, and accuracy of the compressed model, we conduct a controlled ablation study on VGG-16 pretrained on CIFAR-10 to investigate the impact of this hyperparameter. To measure the approximation error between the original and decomposed models, we employ the normalized mean square error (NMSE) defined as:

$$\text{NMSE} = \frac{1}{\sum_{l=1}^L O^{(l)}} \sum_{l=1}^L \sum_{o=1}^{O^{(l)}} \frac{\left\| \mathbf{w}_o^{(l)} - \sum_{b=1}^{B^{(l)}} \mathbf{c}_{ob}^{(l)} \mathbf{B}_b^{(l)} \right\|_F^2}{\left\| \mathbf{w}_o^{(l)} \right\|_F^2}, \quad (32)$$

where $\mathbf{w}_o^{(l)}$ is the o -th 3-order filter in the l -th layer composed of $O^{(l)}$ filters, $\mathbf{B}_b^{(l)}$ and $\mathbf{c}_{ob}^{(l)}$ are respectively the basis filters and the coefficient matrix of the l -th layer, and L is the number of CNN layers. The reductions in MACs and parameters represent the achieved compression ratio. Additionally, we report the accuracy drop of the decomposed model both with and without fine-tuning. Since the number of filters per layer O_l varies, we select the number of basis filters for each layer based on O_l , i.e., $\frac{B^{(l)}}{O^{(l)}} \in \left\{ \frac{1}{8}, \frac{2}{8}, \dots, \frac{7}{8} \right\}$. This discretization makes the choice of $B^{(l)}$ practical, as exhaustively testing all cases ($1 \leq B^{(l)} \leq O^{(l)}$) would be computationally prohibitive.

Table 17 summarizes the compression ratios, approximation errors, and accuracies for different $\frac{B^{(l)}}{O^{(l)}}$ values. The results reveal three clear trends. First, the number of basis filters determines a wide range of compression ratios, from $\sim 1\%$ to 85%. As expected, increasing $B^{(l)}$ reduces the achievable compression, confirming the inverse relationship theoretically analyzed in Subsection 3.2. Second, a higher number of basis filters consistently lowers the NMSE, in agreement with the error bound of the employed SVD (Eq. (15)). Finally, concerning accuracy, we observe that without fine-tuning, our method provides a good balance between accuracy and compression, particularly for moderate settings. For instance, with $\frac{B^{(l)}}{O^{(l)}} = \frac{4}{8}$, the model achieves a 43% parameter reduction with only a 3.14% accuracy drop. However, under very aggressive compression, such as $\frac{B^{(l)}}{O^{(l)}} = \frac{1}{8}$, accuracy deteriorates sharply (−83.69%), indicating insufficient representational capacity of the basis set. Fine-tuning alleviates this limitation: across all tested ratios, the fine-tuned models recover accuracy, in some cases surpassing the original network. This demonstrates that fine-tuning not only compensates for approximation errors introduced by basis sharing but also enhances model stability during training. In particular, settings between $\frac{B^{(l)}}{O^{(l)}} = \frac{2}{8}$ and $\frac{6}{8}$ strike a favorable balance, delivering substantial reductions in MACs and parameters (71–15%) while maintaining high post-finetuning accuracy.

Taken together, these findings highlight that: (i) the number of basis filters directly controls the compression–accuracy trade-off; (ii) the theoretical inverse relationships between $B^{(l)}$ and both compression ratio and approximation error are validated in practice; and (iii) fine-tuning is essential to unlock the full potential of basis sharing, enabling high compression with negligible or even no loss in accuracy.

5.2. Comparison of approximation error against complexity reduction

In Section 4, we compared our approach with related works in terms of accuracy and complexity reduction. While accuracy directly measures the performance of the compressed model, reconstruction error provides complementary insight into the quality of the approximation, even though it is not a deployment-time metric. As introduced in Subsection 3.2 and formalized in Eq. (10), the objective of our basis sharing strategy is to minimize the reconstruction error of the filters. To assess this aspect, we perform an ablation study on a VGG-16 model pretrained on CIFAR-10, focusing on the trade-off between approximation error and complexity reduction. The results, visualized in Fig. 6, report NMSE as a function of both MACs and parameter reduction.

We compare SaP against two representative baselines: NORTON (Pham et al., 2025a), which applies canonical polyadic decomposition to each filter independently, and CoDeC (Pham et al., 2026), which leverages joint tensor decomposition with clustering, effectively introducing weight sharing. For clarity, we describe the results across three regimes of compression—light, moderate, and aggressive—without explicitly labeling them in the figure.

At low reduction levels, up to approximately 30%, all methods yield low NMSE, but SaP consistently achieves the best preservation of the original model. For instance, at around 15% parameter reduction, SaP reaches an NMSE of 0.0275 compared to 0.0372 for NORTON and 0.0531 for CoDeC, corresponding to a relative error reduction of up to 50% compared to CoDeC. This indicates that SaP can provide efficiency gains even under mild compression without noticeable degradation in model fidelity.

In the moderate compression regime, between roughly 30% and 70% reduction, the differences between methods become more pronounced. At about 57% MACs reduction, SaP records an NMSE of 0.1172, while NORTON and CoDeC show higher errors of 0.1837 and 0.1779, respectively. Similarly, at 43% parameter reduction, SaP’s NMSE is 0.0775, compared to 0.1173 for NORTON and 0.1235 for CoDeC. These results illustrate that SaP maintains a smoother increase in error as compression intensifies, whereas the other methods experience a steeper degradation.

Under aggressive compression, above 70% reduction, SaP demonstrates the largest advantage. At reductions exceeding 90%, SaP maintains NMSE below 0.39, whereas NORTON and CoDeC exceed 0.62 and 0.55, respectively. At 85–89% reductions, SaP achieves around 0.29 NMSE, compared to 0.41 for NORTON and 0.36 for CoDeC. This shows that SaP can preserve model fidelity even when most parameters or operations are removed, highlighting its robustness in extreme compression scenarios.

Overall, across all regimes, SaP consistently delivers lower approximation error at equivalent reduction levels. Its gradual error increase and superior stability make it a reliable choice for both conservative and aggressive deployment scenarios.

5.3. On the combination of the number of basis filters and the number of coefficients to be preserved

As a hybrid approach combining weight sharing and coefficient pruning, SaP is influenced by both components. To study their joint effect on the trade-off between model compactness and accuracy, we perform an ablation study on a VGG-16 model pretrained on CIFAR-10. For each layer l with $O^{(l)}$ filters, we uniformly quantize the number of basis filters across all layers, setting $\frac{B^{(l)}}{O^{(l)}} \in \left\{ \frac{1}{8}, \frac{2}{8}, \dots, \frac{7}{8} \right\}$. Similarly, we apply uniform coefficient pruning across layers, with $\frac{P^{(l)}}{O^{(l)}} \in \left\{ \frac{1}{5}, \frac{2}{5}, \dots, \frac{5}{5} \right\}$. This uniform quantization significantly reduces the number of configurations to evaluate, as exploring the full search space ($1 \leq B^{(l)} \leq O^{(l)}$ combined with $1 \leq P^{(l)} \leq O^{(l)}$ across all L layers) would be prohibitively expensive. Fig. 7 shows the results, where each curve corresponds to a fixed ratio $\frac{P^{(l)}}{O^{(l)}}$ and the x-axis varies $\frac{B^{(l)}}{O^{(l)}}$. We report the percentage reduction

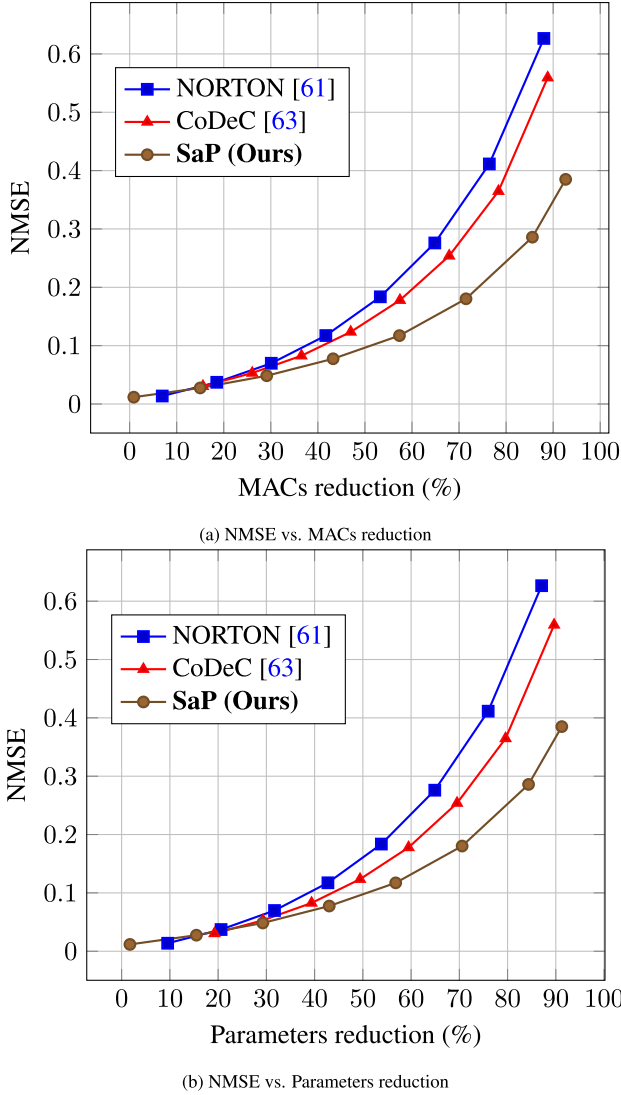


Fig. 6. Comparison of our method, SaP, against NORTON (Pham et al., 2025a) and CoDeC (Pham et al., 2026) in terms of approximation error (NMSE) versus complexity reductions. SaP consistently achieves lower approximation error at higher reduction levels.

in MACs and parameters to quantify compactness, along with the top-1 accuracy drop after fine-tuning to reflect effectiveness.

The results lead to the following observations:

1. The choice of combination of $B^{(l)}$ and $P^{(l)}$ leads to a wide range of both MACs and the number of parameter reductions, ranging from approximately 1% to 99%. This practical example of complexity reduction aligns with the theoretical analysis provided in Subsection 3.4. The flexibility in selecting $B^{(l)}$ and $P^{(l)}$ allows one to choose configurations that match the target complexity reduction.
2. SaP performs effectively when followed by fine-tuning. Across all settings, the post-finetuning accuracy drop remains below 4%. For example, with $\frac{B^{(l)}}{O^{(l)}} = \frac{3}{8}$ and $\frac{P^{(l)}}{O^{(l)}} = \frac{3}{5}$, SaP achieves a 84% reduction in parameters with negligible accuracy loss. In the region where $\frac{B^{(l)}}{O^{(l)}} \geq \frac{4}{8}$ and $\frac{P^{(l)}}{O^{(l)}} \geq \frac{3}{5}$, fine-tuned models even outperform the baseline, suggesting that the uncompressed model is over-parameterized.
3. Increasing either $\frac{B^{(l)}}{O^{(l)}}$ or $\frac{P^{(l)}}{O^{(l)}}$ generally improves accuracy but reduces compression. This trend indicates that more basis filters enhance representational capacity, while retaining more coefficients preserves richer information in reconstructed filters. However, both also in-

crease parameters and computation. Interestingly, the MACs/parameters reduction surface exhibits a non-linear dependency on these two hyperparameters, suggesting that their effects are interdependent rather than additive. Specifically, a moderate number of basis filters combined with a small proportion of preserved coefficients achieves a favorable balance between compactness and representational power. Beyond a certain point, however, increasing P yields diminishing returns, as the additional coefficients contribute marginally to performance recovery while significantly increasing parameter count. Hence, an appropriate balance between these two ratios is crucial to achieve an optimal trade-off between compactness and predictive performance. In practice, moderate values of $\frac{B^{(l)}}{O^{(l)}}$ and $\frac{P^{(l)}}{O^{(l)}}$ yield a favorable compromise, offering strong accuracy with substantial compression.

4. For a given target complexity reduction, multiple combinations of $\frac{B^{(l)}}{O^{(l)}}$ and $\frac{P^{(l)}}{O^{(l)}}$ can produce similar compression levels but different accuracies. For example, to achieve approximately 80% parameter reduction, three distinct configurations, namely $(\frac{2}{8}, \frac{4}{5})$, $(\frac{4}{8}, \frac{3}{5})$, and $(\frac{7}{8}, \frac{2}{5})$, yield reductions of 80.97%, 79.14%, and 83.97%, with accuracy gains of 0.41%, 0.42%, and 0.57%, respectively. These settings correspond to different architectures with varying numbers of basis filters and preserved coefficients per layer. We suggest that determining the optimal $(B^{(l)}, P^{(l)})$ combination for a given compression objective could be formulated as an architecture or hyperparameter search problem. Beyond simple metrics such as reconstruction error or layer sensitivity, more informative criteria like singular value energy retention, Fisher information, or mutual information between inputs and activations could be used to guide selection. Moreover, automated approaches such as Bayesian optimization, reinforcement learning, or Pareto-based multi-objective search could efficiently explore the configuration space, identifying layer-wise combinations that best balance accuracy and efficiency. We leave this as a promising direction for future research.

Overall, this analysis highlights that the interplay between basis sharing and coefficient pruning governs the trade-off between compactness and accuracy in SaP. By tuning $(B^{(l)}, P^{(l)})$, practitioners can flexibly control model complexity, achieving substantial compression with minimal accuracy loss. The observed non-linear interactions between these hyperparameters further suggest that SaP can be adapted to various efficiency targets, making it a versatile framework for model compression.

From a practical perspective, the ablation results suggest a simple and effective guideline for selecting the hyperparameters $(B^{(l)}, P^{(l)})$. According to Eqs. (28) and (29), the overall parameter and MACs reduction ratios depend explicitly on the layer-wise choices of the number of basis filters and preserved coefficients. In general, the number of basis filters primarily controls the representational capacity of the compressed layer, while the number of preserved coefficients determines the level of redundancy removal after basis sharing.

Based on our experiments, selecting $\frac{B^{(l)}}{O^{(l)}} \in [\frac{2}{8}, \frac{6}{8}]$ typically provides a favorable balance between approximation quality and compression, whereas extremely small values (e.g., $\frac{1}{8}$) may lead to insufficient representational capacity under aggressive compression. After choosing $B^{(l)}$, the compression ratio can be further adjusted by tuning the preserved coefficient ratio, where selecting $\frac{P^{(l)}}{O^{(l)}} \leq \frac{3}{5}$ generally achieves substantial parameter reduction while maintaining stable post-finetuning accuracy.

In practice, we therefore recommend a two-stage selection strategy: first determine $B^{(l)}$ to ensure adequate approximation capability, and then adjust $P^{(l)}$ to meet the desired compression target according to the analytical reduction ratios provided in Eqs. (28) and (29). This strategy provides a reliable and interpretable way to control the accuracy-efficiency trade-off when applying SaP to new architectures.

More generally, selecting the optimal layer-wise configuration $\{(B^{(l)}, P^{(l)})\}_{l=1}^L$ for a given compression objective while maximizing

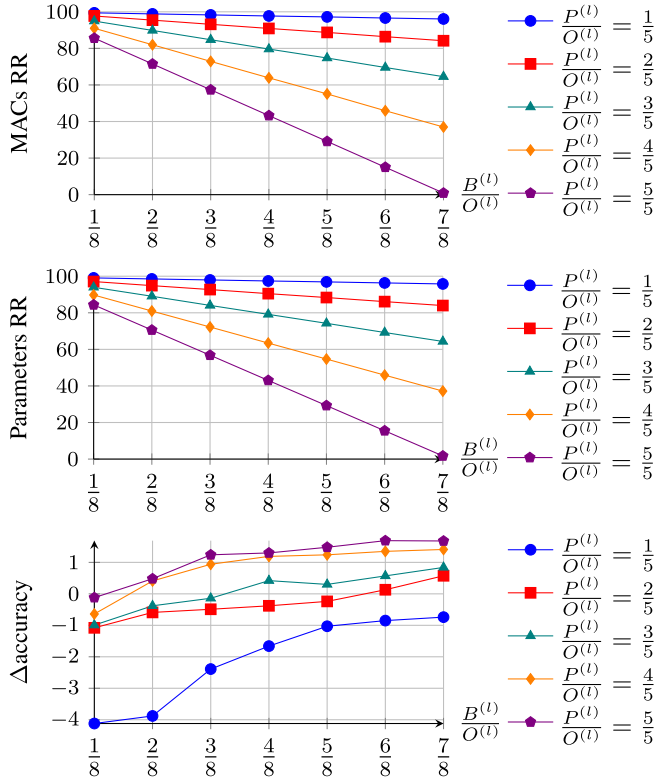


Fig. 7. Impact of the combination between the number of basis filters ($B^{(l)}$) and the number of coefficients preserved ($P^{(l)}$) on model efficiency and accuracy. Each curve corresponds to a fixed ratio $\frac{P^{(l)}}{O^{(l)}}$, while the x-axis varies $\frac{B^{(l)}}{O^{(l)}}$. The results show that increasing either $\frac{B^{(l)}}{O^{(l)}}$ or $\frac{P^{(l)}}{O^{(l)}}$ improves accuracy but reduces compression, highlighting the trade-off between model compactness and performance.

predictive accuracy constitutes a combinatorial optimization problem due to the coupled influence of these parameters across layers. While more sophisticated strategies, such as neural architecture search or automated compression policy learning, could in principle be used to optimize these hyperparameters, they typically introduce substantial computational overhead due to the large search space and repeated model training required. Therefore, the proposed empirical guidelines offer a practical and computationally efficient alternative to exhaustive or search-based optimization when deploying SaP in real-world scenarios.

5.4. On the choice of the kernel bandwidth

In the proposed framework, the kernel bandwidth σ is adaptively determined for each layer based on the pairwise distances among its coefficient vectors. To examine the sensitivity of the method to this parameter, we conducted an ablation study on ResNet-20 trained on CIFAR-10. Specifically, we evaluated five settings for σ : the 25th and 75th percentiles of the pairwise distances, as well as $0.5\times$, $1\times$, and $2\times$ the median distance.

For each configuration, we tested multiple preservation ratios $\frac{P^{(l)}}{O^{(l)}} \in \{\frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, \frac{5}{5}\}$. To isolate the effect of the kernel bandwidth and coefficient pruning, we fixed the ratio of basis filters across all layers to $\frac{B^{(l)}}{O^{(l)}} = \frac{3}{8}$. The impact of the number of basis filters has already been analyzed in Subsections 5.1 and 5.3.

As shown in Fig. 8, the overall accuracy remains remarkably stable across a wide range of kernel bandwidths. Moderate settings—specifically the median and 75th percentile—consistently yield the best performance, whereas excessively narrow kernels (e.g., $0.5\times$ median) tend to slightly degrade accuracy due to oversensitivity to local vari-

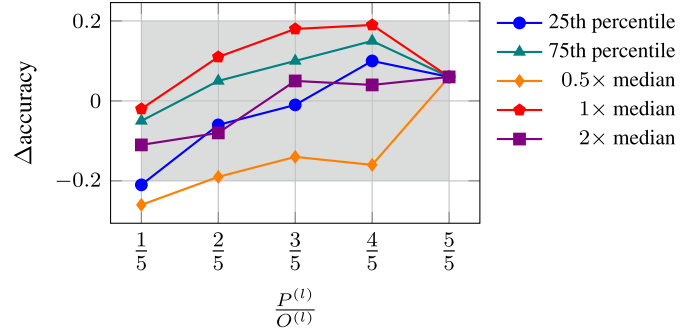


Fig. 8. Impact of the kernel bandwidth σ on the classification performance of ResNet-20 on CIFAR-10, measured as Δ accuracy relative to the median-based σ . Each curve corresponds to a distinct choice of σ , while the x-axis varies the preserved ratio $\frac{P^{(l)}}{O^{(l)}}$. Moderate settings (median and 75th percentile) yield the highest stability, whereas excessively small bandwidths (e.g., $0.5\times$ median) slightly degrade performance. The shaded region ($\pm 0.2\%$) highlights the narrow performance spread across all configurations, confirming the robustness of the adaptive per-layer bandwidth selection.

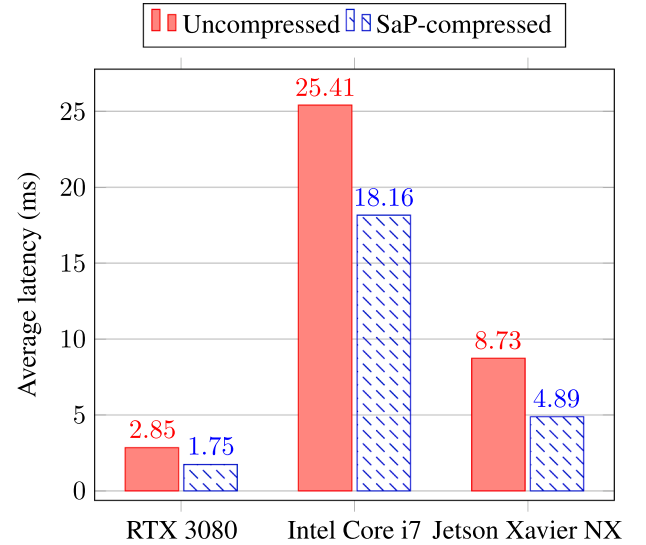


Fig. 9. Average inference latency of the uncompressed and SaP-compressed ResNet-18 across different devices.

ations. In contrast, broader kernels (e.g., $2\times$ median) introduce mild smoothing but remain close to the optimal performance. Across all configurations, the variations are confined within approximately $\pm 0.2\%$, indicating that the adaptive per-layer computation effectively aligns σ with the local feature distribution. This adaptivity eliminates the need for manual tuning, making the method both robust and easy to apply in practice. These empirical observations are consistent with the theoretical discussion of the Gaussian-kernel similarity detailed in Subsection 3.3. While more elaborate optimization of σ could be explored, the current formulation already provides a well-balanced trade-off between sensitivity and stability.

5.5. Comparison of compression overhead under performance

To evaluate the compression efficiency of SaP, we compare it with five representative approaches: DCFF (Lin et al., 2023), White-Box (Zhang et al., 2023), ABCPruner (Lin et al., 2020b), CORING (Pham et al., 2024a), and CoDeC (Pham et al., 2026). Table 18 summarizes the total overhead (compression and fine-tuning time), top-1 accuracy, and reductions in MACs and parameters across five architectures (VGG-16 (Simonyan & Zisserman, 2015), ResNet-56/110 (He et al., 2016),

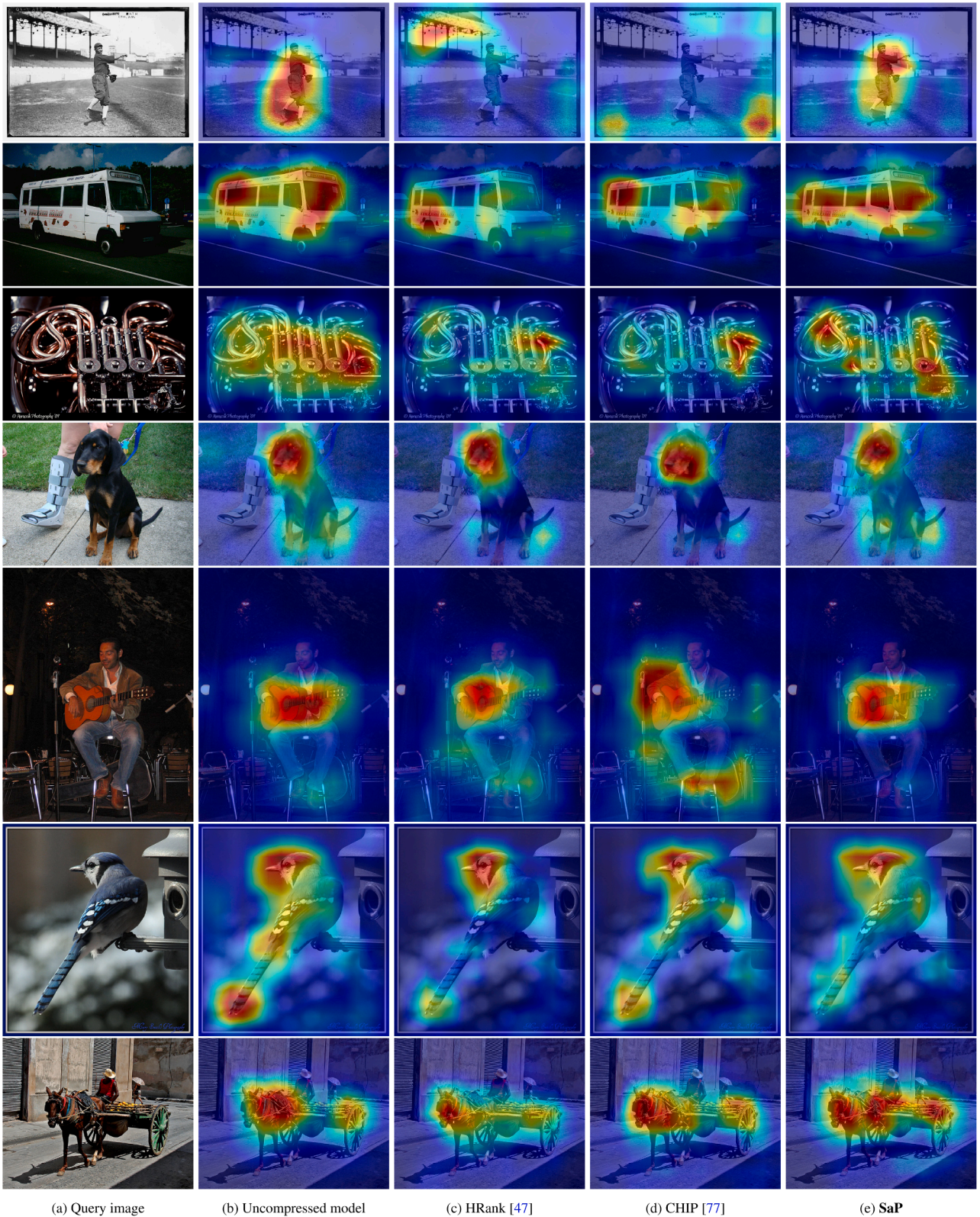


Fig. 10. Feature visualization via GradCAM illustrating that SaP tends to preserve coherent spatial and semantic focus similar to the uncompressed baseline, whereas HRank and CHIP sometimes display partial or shifted attention regions.

Table 18

Comparison of compression overhead under performance.

Model	Dataset	Method	Type	Overhead (seconds)			Accuracy	RR (%)	
				Compress	Finetune	Total		MACs	Params
VGG-16 (Simonyan & Zisserman, 2015)	CIFAR-10	ABCPruner (Lin et al., 2020b)	P + N	5387	1712	7099	93.08	74	89
		CORING (Pham et al., 2024a)	L + P	810	3935	4745	93.68	79	87
		SaP	S + P	402	1539	1941	93.91	80	92
ResNet-56 (He et al., 2016)	CIFAR-10	DCFF (Lin et al., 2023)	P	N/A	4060	4060	93.26	56	55
		SaP	S + P	471	2530	3001	93.82	66	83
ResNet-110 (He et al., 2016)	CIFAR-10	WhiteBox (Zhang et al., 2023)	P	5213	9547	14,760	94.12	65	62
		SaP	S + P	1025	7226	8251	94.43	66	62
GoogLeNet (Szegedy et al., 2015)	CIFAR-10	ABCPruner (Lin et al., 2020b)	P + N	26,968	9881	36,849	94.84	67	60
		DCFF (Lin et al., 2023)	P	N/A	15,180	15,180	94.92	70	66
		SaP	S + P	1568	9487	11055	95.32	77	73
SqueezeNet (Iandola et al., 2016)	CIFAR-100	CoDeC (Pham et al., 2026)	L + S	1009	3846	4855	69.98	31	33
		SaP	S + P	783	2714	3497	70.16	33	34
MobileNetV2 (Sandler et al., 2018)	ImageNet	DCFF (Lin et al., 2023)	P	N/A	261,974	261,974	68.60	53	25
		SaP	S + P	7032	168946	175978	69.75	65	50

GoogLeNet (Szegedy et al., 2015), SqueezeNet (Iandola et al., 2016), and MobileNetV2 (Sandler et al., 2018)) on three datasets (CIFAR-10, CIFAR-100, and ImageNet). All five architectures are among the twelve introduced in Subsection 4.1; as noted there, SqueezeNet is evaluated specifically in this compression-efficiency comparison rather than in the main classification results.

Among pruning-based methods, WhiteBox (Zhang et al., 2023) estimates channel importance from feature maps during training, requiring multiple epochs and resulting in long pruning times. In contrast, SaP performs post-training compression, completing pruning more efficiently and achieving higher accuracy on ResNet-110/CIFAR-10. DCFF (Lin et al., 2023) conducts pruning from scratch, further increasing training time, whereas SaP achieves superior accuracy–efficiency trade-offs on MobileNetV2/ImageNet with substantially shorter compression time.

We further compare SaP with three hybrid methods: ABCPruner (Lin et al., 2020b) (NAS and pruning), CORING (Pham et al., 2024a) (decomposition and pruning), and CoDeC (Pham et al., 2026) (decomposition and parameter sharing). ABCPruner employs an artificial bee colony algorithm to search for compact architectures, but its optimization process is computationally expensive. On VGG-16 and GoogLeNet/CIFAR-10, SaP achieves shorter compression times and better compression ratios. Compared to CORING, which iteratively prunes and fine-tunes decomposed layers, SaP delivers similar accuracy with significantly less compression and fine-tuning time. Against CoDeC, which performs joint tensor decomposition to enable the sharing of common factor matrices, SaP achieves faster compression and higher accuracy on SqueezeNet/CIFAR-100.

Overall, SaP consistently attains competitive or superior compression results across all tested models and datasets, while requiring markedly less time than both pruning-during-training and hybrid approaches. Its efficiency further extends to the fine-tuning stage, where only the lightweight coefficient vectors are updated while the shared bases remain frozen. This design greatly reduces the number of trainable parameters and accelerates convergence, enabling faster and more stable fine-tuning with lower computational cost. Together, these properties underscore the practicality of SaP for time-constrained and resource-limited deployment scenarios.

5.6. Practical latency evaluation

To evaluate the acceleration achieved by SaP, we deploy the uncompressed ResNet-18 model and its SaP-compressed counterpart on a diverse set of hardware platforms that represent a wide range of computational capacities and deployment scenarios. The objective is to assess real-world performance improvements across environments span-

ning from high-performance servers to low-power embedded systems. The selected platforms include a desktop/server-grade GPU (NVIDIA RTX 3080), commonly used for large-scale deep learning tasks; a desktop CPU (Intel Core i7-9700K) representing a standard computing setup; and an embedded device (NVIDIA Jetson Xavier NX), designed for edge computing applications. Each latency was measured for batch size 1, averaged over 1000 runs after 100 warm-up iterations, using device-synchronized timers in PyTorch. The resulting latencies are summarized in Fig. 9.

The results show that SaP not only reduces the theoretical computation (MACs reduced by 49%, see Table 9) but also achieves consistent wall-clock acceleration across heterogeneous hardware. The measured latency speedups are approximately 1.63× on the RTX 3080, 1.40× on the Intel i7 CPU, and 1.81× on the Jetson Xavier NX. These findings confirm that the proposed SaP compression leads to practical inference efficiency gains, particularly for latency-sensitive or resource-constrained deployments. It is worth noting that, although a 49% reduction in MACs corresponds to a theoretical 1.96× computational saving, the observed acceleration is lower in practice due to hardware and software factors such as memory bandwidth limitations and kernel launch overheads. This empirical observation is consistent with recent findings in neural network compression research (Pham et al., 2026).

Beyond the raw speedup figures, these results carry direct practical significance for deployment. On the Jetson Xavier NX, an embedded platform representative of on-device inference in robotics, drones, and mobile vision systems, the measured 1.81× speedup directly increases the achievable frame rate under a fixed power and compute budget, which is typically the limiting factor for meeting real-time responsiveness requirements on such platforms. On the RTX 3080, the 1.63× acceleration translates into higher serving throughput per GPU, reducing the number of GPU-hours required to serve a fixed volume of inference requests in a production setting, and thus lowering energy consumption and infrastructure cost. Even on a commodity CPU without any hardware acceleration, the 1.40× speedup widens the range of deployment scenarios in which SaP-compressed models remain practical without requiring a dedicated accelerator. Independently of these inference-time gains, the accompanying parameter reduction directly lowers the on-disk and in-memory footprint of the model, which is frequently the binding constraint for over-the-air updates and on-device storage on embedded and mobile hardware. Taken together, these platform-level results indicate that the compression achieved by SaP translates into measurable, deployment-relevant benefits across the spectrum of hardware budgets encountered in practice, from cloud-scale serving to embedded edge devices, rather than remaining a favorable number on a theoretical FLOPs count alone.

Table 19
Stability of SaP fine-tuning across multiple random seeds.

Model	Dataset	Seeds	Mean	Standard deviation
VGG-16	CIFAR-10	{0,1,2,3,4}	93.92	0.08
ResNet-32	CIFAR-100	{0,1,2,3,4}	72.41	0.05
MobileNetV2	ImageNet	{0,1}	69.75	0.02

5.7. Statistical robustness

In SaP, the decomposition and pruning stages are fully deterministic, while only the fine-tuning process involves stochasticity due to stochastic optimization and data shuffling. To examine how this randomness might affect performance, we conducted additional fine-tuning experiments on CIFAR-10, CIFAR-100, and ImageNet using multiple random seeds. The results are summarized in Table 19.

Overall, SaP demonstrates stable performance across different random initializations. The top-1 accuracies show minimal variation across seeds; for instance, VGG-16 on CIFAR-10 achieves an average accuracy of 93.92% with a standard deviation of 0.08%. These findings support the representativeness of the single-run results reported in Section 4. Nevertheless, we acknowledge that other factors, such as choices of optimizer, learning rate scheduler, or other hyperparameter settings, may also influence the final outcomes; however, these aspects are beyond the scope of the present study.

5.8. Interpretability analysis via feature visualization

To further understand how SaP preserves discriminative information after compression, we conduct a qualitative analysis of feature activation maps using GradCAM (Selvaraju et al., 2020). This visualization assesses whether the compressed networks maintain attention on the same semantic regions as the uncompressed baseline. We compare SaP with two representative pruning-based compression approaches, HRank (Lin et al., 2020a) and CHIP (Sui et al., 2021), under similar compression ratios (approximately 71% parameter and 77% MACs reduction) using ResNet-50 on ImageNet (see Table 11).

As illustrated in Fig. 10, SaP consistently preserves the spatial focus and semantic coherence of feature activations across various object categories. For instance, with the *guitar* class, SaP retains all the necessary features of the complete object. In contrast, HRank only captures the body of the guitar, while CHIP focuses on the lower bout and irrelevant elements such as the artist's arm. The difference becomes even more apparent in the *ball player* samples, where SaP closely mirrors the baseline activation pattern, attending to the player's full silhouette and throwing posture. In contrast, HRank and CHIP display misaligned attention: HRank overemphasizes the background, whereas CHIP shifts focus toward irrelevant ground regions, which indicates a partial loss of task-relevant features. These observations demonstrate SaP's superior ability to retain class-specific and contextually relevant information, even under substantial compression.

These results suggest that the orthogonal basis decomposition in SaP preserves diverse yet discriminative feature subspaces, while the redundancy-aware coefficient pruning effectively removes only non-informative components. Consequently, the compressed network continues to activate on task-relevant regions with minimal spatial drift. The close correspondence between the baseline and SaP activation maps confirms that our approach achieves structural compactness without compromising representational integrity.

Overall, these qualitative findings complement the quantitative results presented in Section 4, reinforcing that SaP preserves both global and local feature representations more faithfully than existing methods. This feature integrity directly contributes to the observed robustness and accuracy retention in highly compressed models.

6. Conclusion

In this work, we introduced SaP, a unified compression framework that jointly integrates basis sharing and coefficient pruning to address redundancy in both filter and feature spaces of convolutional neural networks. By representing filters as linear combinations of a reduced set of shared basis filters and subsequently pruning redundant coefficients, SaP achieves a more compact and computationally efficient representation while preserving the discriminative capacity of the original model.

Through extensive experiments on a wide range of architectures, datasets, and tasks, we demonstrated that SaP consistently outperforms state-of-the-art compression approaches in terms of accuracy retention, compression ratio, and practical efficiency. Theoretical analysis and empirical studies further confirm that the proposed hybrid formulation alleviates the bottleneck issues inherent in basis-sharing methods and provides fine-grained control over model compactness via coefficient pruning. Moreover, the orthonormality of the shared bases allows efficient redundancy detection and pruning directly in the coefficient subspace, substantially reducing computational overhead during fine-tuning.

Beyond its quantitative advantages, SaP offers conceptual insights into the complementary nature of weight sharing and pruning. By bridging these two traditionally separate paradigms within a single formulation, SaP establishes a new direction for designing efficient neural architectures that balance structural compactness, training simplicity, and hardware deployability.

Overall, the results and analyses presented in this study underscore the effectiveness of combining complementary compression mechanisms and open new avenues toward practical, high-performance model compression for deep neural networks.

Limitations. Despite these advantages, SaP has several limitations that merit discussion. First, the layer-wise hyperparameters ($B^{(l)}$, $P^{(l)}$) are selected using the empirical guidelines derived from the grid-based ablation study in Subsection 5.3 rather than an automated search procedure; while effective and computationally cheap in practice, this heuristic is not guaranteed to identify the globally optimal configuration for every architecture. Second, due to the computational cost of repeated large-scale training, the main results on ImageNet and MS-COCO are reported from a single representative run; our robustness analysis (Subsection 5.7) shows negligible seed-to-seed variation on the settings we could afford to repeat, but broader statistical validation at this scale remains an open direction. Third, our evaluation on transformer-based and hybrid CNN-transformer architectures (Subsubsection 4.4.6 and Subsubsection 4.5.2) applies basis sharing and coefficient pruning to linear projection layers without explicitly modeling attention-specific redundancy, such as head-level or token-level redundancy (Pham & Le, 2026); extending the framework to exploit these structures is a promising direction for future work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

All datasets used in this work (CIFAR-10, CIFAR-100, ImageNet, and MS-COCO 2017) are public benchmarks. Source code and pretrained models will be made publicly available upon acceptance of this paper.

References

- Chang, J., Lu, Y., Xue, P., Xu, Y., & Wei, Z. (2023). Iterative clustering pruning for convolutional neural networks. *Knowledge-Based Systems*, 265, 110386.
- Chen, D., Lin, K., & Deng, Q. (2024a). Gap: A group-based automatic pruning algorithm via convolution kernel fusion. *Neurocomputing*, 609, 128488.

- Chen, S., Zhou, J., Sun, W., & Huang, L. (2023). Joint matrix decomposition for deep convolutional neural networks compression. *Neurocomputing*, 516, 11–26.
- Chen, T., Cheng, Y., Gan, Z., Yuan, L., Zhang, L., & Wang, Z. (2021). Chasing sparsity in vision transformers: An end-to-end exploration. In *Advances in neural information processing systems* (pp. 19974–19988). (vol. 34).
- Chen, X., Liu, C., Hu, P., Lin, J., Gong, Y., Chen, Y., Peng, D., & Geng, X. (2024b). Evolving filter criteria for randomly initialized network pruning in image classification. *Neurocomputing*, (p. 127872). <https://doi.org/10.1016/j.neucom.2024.127872>
- Chen, Y., Wen, X., Zhang, Y., & He, Q. (2022). Fpc: Filter pruning via the contribution of output feature map for deep convolutional neural networks acceleration. *Knowledge-Based Systems*, 238, 107876.
- Chen, Z., Xiang, J., Lu, Y., Xuan, Q., Wang, Z., Chen, G., & Yang, X. (2024c). Rgp: Neural network pruning through regular graph with edges swapping. *IEEE Transactions on Neural Networks and Learning Systems*, 35(10), 14671–14683.
- Dai, C., Liu, X., Cheng, H., Yang, L. T., & Deen, M. J. (2021). Compressing deep model with pruning and tucker decomposition for smart embedded systems. *IEEE Internet of Things Journal*, 9(16), 14490–14500.
- Dai, W., Fan, J., Miao, Y., & Hwang, K. (2025). Deep learning model compression with rank reduction in tensor decomposition. *IEEE Transactions on Neural Networks and Learning Systems*, 36(1), 1315–1328. <https://doi.org/10.1109/TNNLS.2023.3330542>
- De Jonghe, J., Pham, V. T., & Ishteva, M. (2026). Robust basis spline decoupling for the compression of transformer models. arXiv preprint arXiv:2605.18794
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houshy, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International conference on learning representations* (pp. 1–21).
- Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.
- Fan, C., Li, J., Zhang, Z., Li, F., & Wang, B. (2026). Two-phase collaborative model compression training for joint pruning and quantization. *Neural Networks*, 197, 108506.
- Gabor, M., & Zdunek, R. (2026). Reduced storage direct tensor ring decomposition for convolutional neural networks compression. *Neural Networks*, 193, 107994.
- Ganjanesh, A., Gao, S., & Huang, H. (2024). Jointly training and pruning cnns via learnable agent guidance and alignment. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 16058–16069).
- Gao, C., Cao, Q., Yao, X., Feng, X., & Cheng, G. (2026). Dual-perspective filter pruning via diversity and independence collaboration. *Pattern Recognition*, 177, 113321.
- Gao, S., Zhang, Y., Huang, F., & Huang, H. (2024). Bilevelpruning: Unified dynamic and static channel pruning for convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 16090–16100).
- Guan, Y., Liu, N., Zhao, P., Che, Z., Bian, K., Wang, Y., & Tang, J. (2023). Dais: Automatic channel pruning via differentiable annealing indicator search. *IEEE Transactions on Neural Networks and Learning Systems*, 34(12), 9847–9858. <https://doi.org/10.1109/TNNLS.2022.3161284>
- Guo, K., Lin, Z., Chen, C., Xing, X., Liu, F., & Xu, X. (2025). Compact model training by low-rank projection with energy transfer. *IEEE Transactions on Neural Networks and Learning Systems*, 36(4), 6708–6722. <https://doi.org/10.1109/TNNLS.2024.3400928>
- Guo, Y., Gao, W., & Li, G. (2024). Interpretable task-inspired adaptive filter pruning for neural networks under multiple constraints. *International Journal of Computer Vision*, 132(6), 2060–2076.
- Hajimolahoseini, H., Ahmed, W., Rezagholizadeh, M., Partovinia, V., & Liu, Y. (2022). Strategies for applying low rank decomposition to transformer-based models. In *36th conference on neural information processing systems (neurIPS2022)* (pp. 1–6).
- Halko, N., Martinsson, P.-G., & Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2), 217–288.
- He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2020). Mask r-CNN. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(2), 386–397. <https://doi.org/10.1109/TPAMI.2018.2844175>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).
- Hou, Y., Ma, Z., Liu, C., Wang, Z., & Loy, C. C. (2024). Network pruning via resource re-allocation. *Pattern Recognition*, 145, 109886. <https://doi.org/10.1016/j.patcog.2023.109886>
- Hu, W., Xiao, L., & Pennington, J. (2020). Provable benefit of orthogonal initialization in optimizing deep linear networks. In *International conference on learning representations*.
- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4700–4708).
- Iandola, F. N., Han, S., Moskewicz, M. W., Ashraf, K., Dally, W. J., & Keutzer, K. (2016). Squeezenet: Alexnet-level accuracy with 50x fewer parameters and < 0.5 MB model size. arXiv preprint arXiv:1602.07360
- Jayasimhan, A., & Pabitha, P. (2024). Resprune: An energy-efficient restorative filter pruning method using stochastic optimization for accelerating CNN. *Pattern Recognition*, (p. 110671).
- Ji, F., Chen, X., Chu, R., & Liu, B. (2025). Network slimming using lp ($p < 1$) regularization. *Pattern Recognition*, (p. 111711).
- Kang, W., & Kim, D. (2021). Deeply shared filter bases for parameter-efficient convolutional neural networks. *Advances in Neural Information Processing Systems*, 34, 7397–7408.
- Kolda, T. G., & Bader, B. W. (2009). Tensor decompositions and applications. *SIAM Review*, 51(3), 455–500.
- Krizhevsky, A., Hinton, G. et al. (2009). Learning multiple layers of features from tiny images.
- Kumar, A., Shibu, A., & Lee, D.-G. (2025). Automatic network compression via evolutionary search-guided channel pruning of deep convolutional neural networks. *Applied Soft Computing*, (p. 113521).
- Lee, K., & Lee, H.-W. (2026). Dynamic network compression via probabilistic channel pruning. *Neural Networks*, 193, 108080.
- Lee, S., & Song, B. C. (2024). Fast filter pruning via coarse-to-fine neural architecture search and contrastive knowledge transfer. *IEEE Transactions on Neural Networks and Learning Systems*, 35(7), 9674–9685. <https://doi.org/10.1109/TNNLS.2023.3236336>
- Li, G., Li, R., Li, T., Shen, C., Zou, X., Wang, J., Wang, C., & Li, N. (2025). Sfp: Similarity-based filter pruning for deep neural networks. *Information Sciences*, 689, 121418.
- Li, G., Zhang, M., Wang, J., Weng, D., & Corporaal, H. (2022). Scwc: Structured channel weight sharing to compress convolutional neural networks. *Information Sciences*, 587, 82–96.
- Li, J., Shao, H., Deng, X., & Jiang, Y. (2024). Efficient filter pruning: Reducing model complexity through redundancy graph decomposition. *Neurocomputing*, (p. 128108).
- Li, Q., Chen, Z., Li, Y., Jiang, Z., & Cao, S. (2026). Efficient network compression via gradient-score aware pruning. *Neurocomputing*, 660, 131870.
- Li, S., Hanson, E., Li, H., & Chen, Y. (2020a). Penni: Pruned kernel sharing for efficient cnn inference. In *International conference on machine learning* (pp. 5863–5873). PMLR.
- Li, Y., Gu, S., Gool, L. V., & Timofte, R. (2019). Learning filter basis for convolutional neural network compression. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 5623–5632).
- Li, Y., Gu, S., Mayer, C., Van Gool, L., & Timofte, R. (2020b). Group sparsity: The hinge between filter pruning and decomposition for network compression. In *2020 IEEE/CVF Conference on computer vision and pattern recognition (CVPR)* (pp. 8015–8024). <https://doi.org/10.1109/CVPR42600.2020.00804>
- Liebenwein, L., Maalouf, A., Feldman, D., & Rus, D. (2021). Compressing neural networks: Towards determining the optimal layer-wise decomposition. *Advances in Neural Information Processing Systems*, 34, 5328–5344.
- Lin, H., Peng, Y., Zhang, Y., Bie, L., Zhao, X., & Gao, Y. (2025). Filter pruning by high-order spectral clustering. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(4), 2402–2415. <https://doi.org/10.1109/TPAMI.2024.3524381>
- Lin, M., Chen, B., Chao, F., & Ji, R. (2023). Training compact CNNs for image classification using dynamic-coded filter fusion. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(8), 10478–10487. <https://doi.org/10.1109/TPAMI.2023.3259402>
- Lin, M., Ji, R., Wang, Y., Zhang, Y., Zhang, B., Tian, Y., & Shao, L. (2020a). Hrank: Filter pruning using high-rank feature map. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 1529–1538).
- Lin, M., Ji, R., Zhang, Y., Zhang, B., Wu, Y., & Tian, Y. (2020b). Channel pruning via automatic structural search. In *Proceedings of the twenty-ninth international joint conference on artificial intelligence, IJCAI-20* (pp. 673–679).
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft coco: Common objects in context. In *European conference on computer vision* (pp. 740–755). Springer.
- Liu, J., Tang, D., Huang, Y., Zhang, L., Zeng, X., Li, D., Lu, M., Peng, J., Wang, Y., Jiang, F. et al. (2024a). Updp: A unified progressive depth pruner for cnn and vision transformer. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 13891–13899). (vol. 38).
- Liu, J., Zhuang, B., Zhuang, Z., Guo, Y., Huang, J., Zhu, J., & Tan, M. (2022). Discrimination-aware network pruning for deep model compression. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(8), 4035–4051. <https://doi.org/10.1109/TPAMI.2021.3066410>
- Liu, Y., Fan, K., Wu, D., & Zhou, W. (2023). Filter pruning by quantifying feature similarity and entropy of feature maps. *Neurocomputing*, 544, 126297. <https://doi.org/10.1016/j.neucom.2023.126297>
- Liu, Y., Fan, K., & Zhou, W. (2024b). Fpwt: Filter pruning via wavelet transform for cnns. *Neural Networks*, 179, 106577.
- Lu, X., Dong, W., Li, X., Wu, J., Li, L., & Shi, G. (2023). Adaptive search-and-training for robust and efficient network pruning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(8), 9325–9338. <https://doi.org/10.1109/TPAMI.2023.3248612>
- Niu, S., Gao, K., Ma, P., Gao, X., Zhao, H., Dong, J., Chen, Y., & Shen, D. (2023). Exploiting sparse self-representation and particle swarm optimization for CNN compression. *IEEE Transactions on Neural Networks and Learning Systems*, 34(12), 10266–10278.
- Pham, V. T. (2025). Deep Neural Network Compression Using Pruning and Low-Rank Approximations. Ph.D. thesis. Université de Toulon. <https://hal.science/tel-05538668>.
- Pham, V. T., & Le, T. T. (2026). Structure-aware global rank allocation for low-rank compression of dual-tower vision-language models. <https://ssrn.com/abstract=7417408>.
- Pham, V. T., Zniyed, Y., & Nguyen, T. P. (2023). Élagage efficace des filtres basé sur les décompositions tensorielles. In *Xixieme colloque francophone de traitement du signal et des images, gretsi-groupe de recherche en traitement du signal et des images* (pp. 937–940).
- Pham, V. T., Zniyed, Y., & Nguyen, T. P. (2024a). Efficient tensor decomposition-based filter pruning. *Neural Networks*, (p. 106393). <https://doi.org/10.1016/j.neunet.2024.106393>
- Pham, V. T., Zniyed, Y., & Nguyen, T. P. (2024b). Hybrid network compression through tensor decompositions and pruning. In *2024 32nd European signal processing conference (EUSIPCO)* (pp. 1052–1056). IEEE.
- Pham, V. T., Zniyed, Y., & Nguyen, T. P. (2025a). Enhanced network compression through tensor decompositions and pruning. *IEEE Transactions on Neural Networks and Learning Systems*, 36(3), 4358–4370. <https://doi.org/10.1109/TNNLS.2024.3370294>
- Pham, V. T., Zniyed, Y., & Nguyen, T. P. (2025b). Singular values-driven automated filter pruning. *Neural Networks*, (p. 107857).
- Pham, V. T., Zniyed, Y., & Nguyen, T. P. (2026). Coupled tensor decomposition for compact network representation. *IEEE Transactions on Neural Networks and Learning Systems*, 37(2), 617–631.

- Phan, A.-H., Ermilov, D., Kozyrskiy, N., Vorona, I., & Sobolev, K. (2024). How to train your unstable looped tensor network. *IEEE Journal of Selected Topics in Signal Processing*, 18(6), 1036–1045. <https://doi.org/10.1109/JSTSP.2024.3463480>
- Qian, Y., He, Z., Wang, Y., Wang, B., Ling, X., Gu, Z., Wang, H., Zeng, S., & Swaileh, W. (2024). Hierarchical threshold pruning based on uniform response criterion. *IEEE Transactions on Neural Networks and Learning Systems*, 35(8), 10869–10881.
- Ravi Ganesh, M., Blanchard, D., Corso, J. J., & Sekeh, S. Y. (2024). Slimming neural networks using adaptive connectivity scores. *IEEE Transactions on Neural Networks and Learning Systems*, 35(3), 3794–3808. <https://doi.org/10.1109/TNNLS.2022.3198580>
- Ren, S., He, K., Girshick, R., & Sun, J. (2017). Faster r-CNN: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6), 1137–1149. <https://doi.org/10.1109/TPAMI.2016.2577031>
- Ruan, X., Liu, Y., Yuan, C., Li, B., Hu, W., Li, Y., & Maybank, S. (2021). Edp: An efficient decomposition and pruning scheme for convolutional neural network compression. *IEEE Transactions on Neural Networks and Learning Systems*, 32(10), 4499–4513. <https://doi.org/10.1109/TNNLS.2020.3018177>
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M. et al. (2015). Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3), 211–252.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4510–4520).
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2020). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, 128(2), 336–359. <https://doi.org/10.1007/s11263-019-01228-7>
- Shi, M., Liu, C., Jiao, J., & Ye, Q. (2024). Self-supervised feature-gate coupling for dynamic network pruning. *Pattern Recognition*, (p. 110594). <https://doi.org/10.1016/j.patcog.2024.110594>
- Shin, E., Cho, I., Muhammad, A., Shahab Uddin, A. F. M., Jang, Y., & Bae, S.-H. (2024). G-SHARP: Globally shared kernel with pruning for efficient CNNs. In *Icassp 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 5955–5959). <https://doi.org/10.1109/ICASSP48485.2024.10446936>
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *3rd international conference on learning representations* (pp. 1–14).
- Su, X., You, S., Xie, J., Wang, F., Qian, C., Zhang, C., & Xu, C. (2023). Searching for network width with bilaterally coupled network. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(7), 8936–8953. <https://doi.org/10.1109/TPAMI.2022.3226777>
- Sui, Y., Yin, M., Gong, Y., & Yuan, B. (2025). Co-exploring structured sparsification and low-rank tensor decomposition for compact DNNs. *IEEE Transactions on Neural Networks and Learning Systems*, 36(4), 6642–6654. <https://doi.org/10.1109/TNNLS.2024.3408099>
- Sui, Y., Yin, M., Xie, Y., Phan, H., Aliari Zonouz, S., & Yuan, B. (2021). Chip: Channel independence-based pruning for compact neural networks. *Advances in Neural Information Processing Systems*, 34, 24604–24616.
- Sun, W., Chen, S., Huang, L., So, H. C., & Xie, M. (2021). Deep convolutional neural network compression via coupled tensor decomposition. *IEEE Journal of Selected Topics in Signal Processing*, 15(3), 603–616. <https://doi.org/10.1109/JSTSP.2020.3038227>
- Sun, X., & Shi, H. (2024). Towards better structured pruning saliency by reorganizing convolution. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision* (pp. 2204–2214).
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). Going deeper with convolutions. In *2015 IEEE Conference on computer vision and pattern recognition (CVPR)* (pp. 1–9). <https://doi.org/10.1109/CVPR.2015.7298594>
- Tan, M., & Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th international conference on machine learning* (pp. 6105–6114). (vol. 97).
- Tang, X., Ye, S., Shi, Y., Hu, T., Peng, Q., & You, X. (2025). Filter pruning based on information capacity and independence. *IEEE Transactions on Neural Networks and Learning Systems*, 36(5), 8401–8413. <https://doi.org/10.1109/TNNLS.2024.3415068>
- Tofigh, S., Askarizadeh, M., Ahmad, M. O., Swamy, M. N. S., & Nguyen, K. K. (2026a). Acli: A cnn pruning framework leveraging adjacent convolutional layer interdependence and γ -weakly submodularity. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 48(1), 932–945. <https://doi.org/10.1109/TPAMI.2025.3610113>
- Tofigh, S., Askarizadeh, M., Omair Ahmad, M., Swamy, M. N. S., & Nguyen, K. K. (2026b). Resource-efficient and layer interdependence-aware CNN pruning leveraging filter replacement. *IEEE Transactions on Neural Networks and Learning Systems*, (pp. 1–14). <https://doi.org/10.1109/TNNLS.2026.3658318>
- Tokcan, N., Sofi, S. S., Pham, V. T., Prévost, C., Kharbech, S., Magnier, B., Nguyen, T. P., Zniyed, Y., & De Lathauwer, L. (2026). Tensor decompositions for signal processing: Theory, advances, and applications. *Signal Processing*, 238, 110191.
- Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., & Jégou, H. (2021). Training data-efficient image transformers & distillation through attention. In *International conference on machine learning* (pp. 10347–10357).
- Tu, K.-T., Yu, P.-H., Tseng, Y.-S., & Chien, S.-Y. (2025). Fgfp: A fractional gaussian filter and pruning for deep neural networks compression. In *International conference on machine learning* (pp. 1–8).
- Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2023). Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 7464–7475).
- Wong, K. C. L., Kashyap, S., & Moradi, M. (2024). Basis scaling and double pruning for efficient inference in network-based transfer learning. *Pattern Recognition Letters*, 177, 1–6.
- Wu, X., Gao, S., Zhang, Z., Li, Z., Bao, R., Zhang, Y., Wang, X., & Huang, H. (2024). Auto-train-once: Controller network guided automatic network pruning from scratch. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 16163–16173).
- Xiao, J., Zhang, C., Gong, Y., Yin, M., Sui, Y., Xiang, L., Tao, D., & Yuan, B. (2023). Haloc: Hardware-aware automatic low-rank compression for compact neural networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 10464–10472). (vol. 37).
- Yang, L., Gu, S., Shen, C., Zhao, X., & Hu, Q. (2024). Soft independence guided filter pruning. *Pattern Recognition*, 153, 110488.
- Yang, Y., Yu, J., Jovic, N., Huan, J., & Huang, T. S. (2020). Fsnets: Compression of deep convolutional neural networks by filter summary. In *International conference on learning representations* (pp. 1–12).
- Yang, Z., Wang, Y., Liu, C., Chen, H., Xu, C., Shi, B., Xu, C., & Xu, C. (2019). Legonet: Efficient convolutional neural networks with lego filters. In *International conference on machine learning* (pp. 7005–7014). PMLR.
- Yuan, T., Li, Z., Liu, B., Tang, Y., & Liu, Y. (2024). Arpruning: An automatic channel pruning based on attention map ranking. *Neural Networks*, (p. 106220).
- Zhang, H., & Liu, Y. (2025). Bfp: Balanced filter pruning via knowledge distillation for efficient deployment of cnns on edge devices. *Neurocomputing*, (p. 130946).
- Zhang, Y., & Freris, N. M. (2024). Adaptive filter pruning via sensitivity feedback. *IEEE Transactions on Neural Networks and Learning Systems*, 35(8), 10996–11008.
- Zhang, Y., Lin, M., Lin, C.-W., Chen, J., Wu, Y., Tian, Y., & Ji, R. (2023). Carrying out CNN channel pruning in a white box. *IEEE Transactions on Neural Networks and Learning Systems*, 34(10), 7946–7955.
- Zhang, Z., Cao, C., Zheng, F., Sun, T., & Zhao, L. (2025). Gcpnet: Gradient-aware channel pruning network with bilateral coupled sampling strategy. *Expert Systems with Applications*, 266, 126104.
- Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., & Chen, J. (2024). Detsr beat yolos on real-time object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 16965–16974).